



COMP3011

计算机体系结构

3.2 GPU和CUDA编程

授课教师：王强

邮箱：qiang.wang@hit.edu.cn



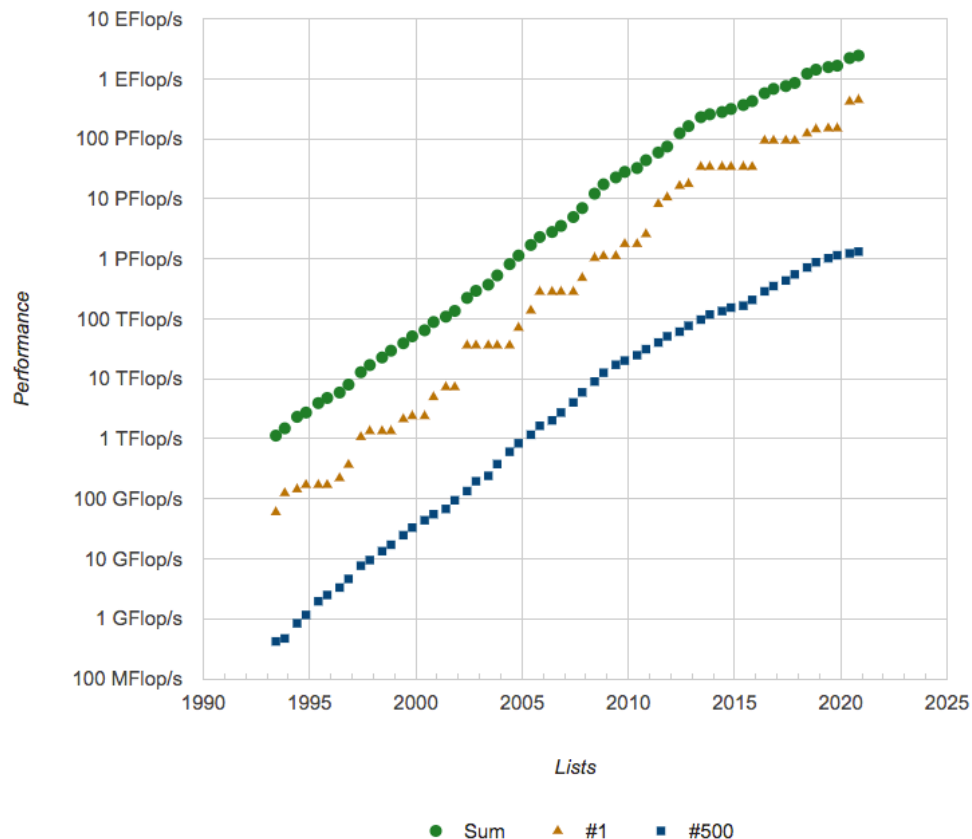
目录 | CONTENT

- 1 GPU架构与编程模型
- 2 CUDA编程
- 3 GPU访存优化
- 4 GPU控制流和指令优化



TOP500算力榜

Performance Development



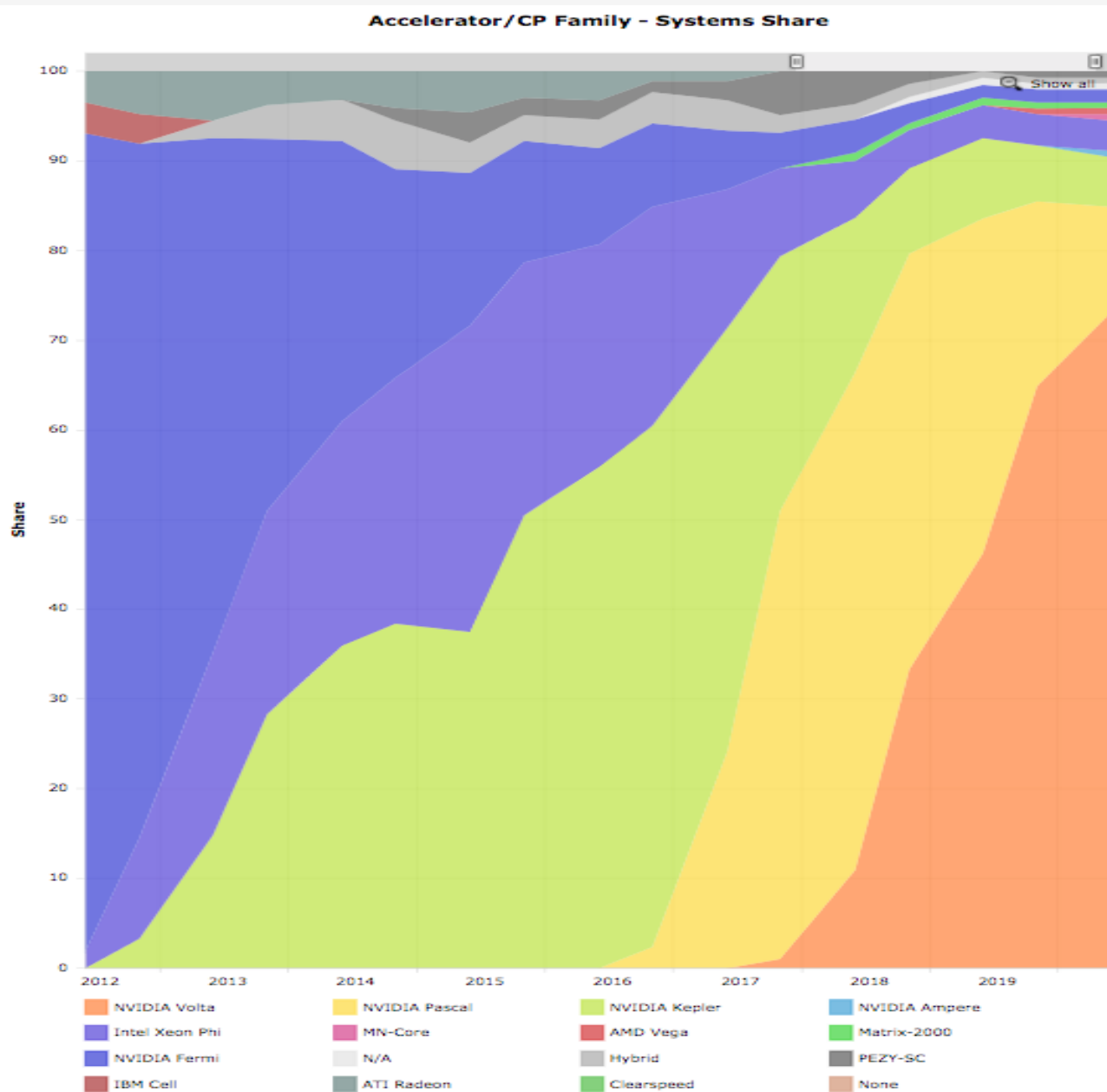
- 图形处理单元 (GPUs) 在超算中扮演重要角色

Rank	System	Cores	Rmax (TFlop/s)	Rpeak (TFlop/s)	Power (kW)
1	Supercomputer Fugaku - Supercomputer Fugaku, A64FX 48C 2.26GHz, Tofu Interconnect D, Fujitsu RIKEN Center for Computational Science Japan	7,630,848	442,010.0	537,212.0	29,899
2	Summit - IBM Power System AC922, IBM POWER9 22C 3.07GHz, NVIDIA Volta GV100, Dual-rail Mellanox EDR Infiniband, IBM DOE/SC/Oak Ridge National Laboratory United States	2,414,592	148,600.0	200,794.9	10,096
3	Sierra - IBM Power System AC922, IBM POWER9 22C 3.1GHz, NVIDIA Volta GV100, Dual-rail Mellanox EDR Infiniband, IBM / NVIDIA / Mellanox DOE/NNSA/LLNL United States	1,572,480	94,640.0	125,712.0	7,438
4	Sunway TaihuLight - Sunway MPP, Sunway SW26010 260C 1.45GHz, Sunway, NRCPC National Supercomputing Center in Wuxi China	10,649,600	93,014.6	125,435.9	15,371
5	Perlmutter - HPE Cray EX235n, AMD EPYC 7763 64C 2.45GHz, NVIDIA A100 SXM4 40 GB, Slingshot-10, HPE DOE/SC/LBNL/NERSC United States	761,856	70,870.0	93,750.0	2,589
6	Selene - NVIDIA DGX A100, AMD EPYC 7742 64C 2.25GHz, NVIDIA A100, Mellanox HDR Infiniband, Nvidia NVIDIA Corporation United States	555,520	63,460.0	79,215.0	2,646

Source: <https://www.top500.org/statistics/perfdevel/>

TOP500算力榜

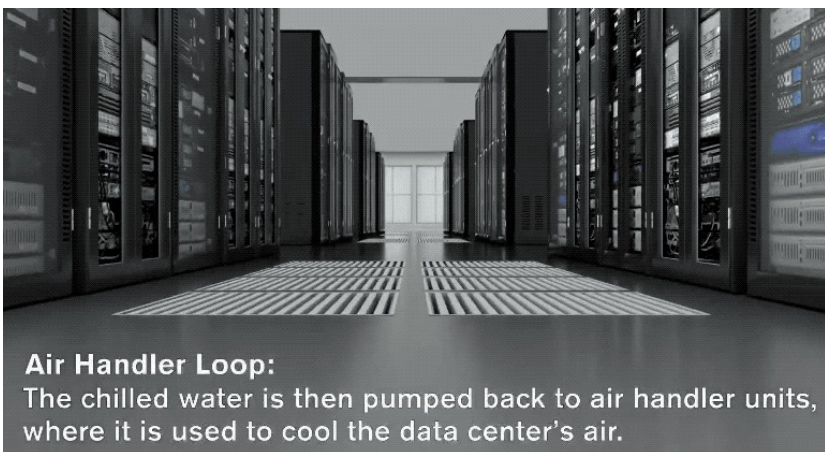
- Top500中约92%的超算系统配备了GPU (June 2020)



人工智能与高性能计算



- 大语言模型的参数量：
 - GPT-3: 1750亿
 - Llama-2: 700亿
- 2023年2月, ChatGPT 吸引了全球 16 亿次访问。
- 按照一万张Nvidia A100来算的话, 运行一个月就要用掉 585 万度电, 大概与我们 8 万人用电相当。
- 散热方面, 利用蒸发水来散热, 但运行起来需要消耗大量的清水, 一个用户与 ChatGPT 进行 25~50 个问题的对话, 大概就相当于请 ChatGPT 喝了 500ml 水。



AI 不止费显卡, 住恒温的大 house, 胃口还出奇的好, 大口吃电, 大口喝水。



- 2002 年之前，通过提高处理器的工作频率实现处理器性能的提升
- 2002年之后，处理器频率保持在 2GHz - 4GHz 之间
 - 处理器面临散热问题
 - 主要通过引入并行性提高性能
 - 多核心
 - SIMD(如MMX, SSE, AVX等指令集)



- 多核 CPU 将两个或多个独立内核整合到一个封装中
 - IBM 于 2001 年推出的 POWER4 在单个芯片上集成了两个完整的 CPU 内核。
 - POWER5, POWER6, POWER7 (4-8核)
 - Sun 于 2005 年发布了配备 8 个内核的 Niagara UltraSPARC 处理器，并于 2010 年发布了配备 16 个内核的 SPARC T3 处理器
- 英特尔和 AMD 于 2005 年推出了双核处理器
 - 2014 英特尔于 2006 年推出 4 核处理器，2010 年推出 8 核处理器，2011 年推出 10 核处理器，2014 年推出 18 核处理器
 - AMD 于 2007 年发布 4 核处理器，2010 年发布 12 核处理器，2014 年发布 16 核处理器



众核处理器

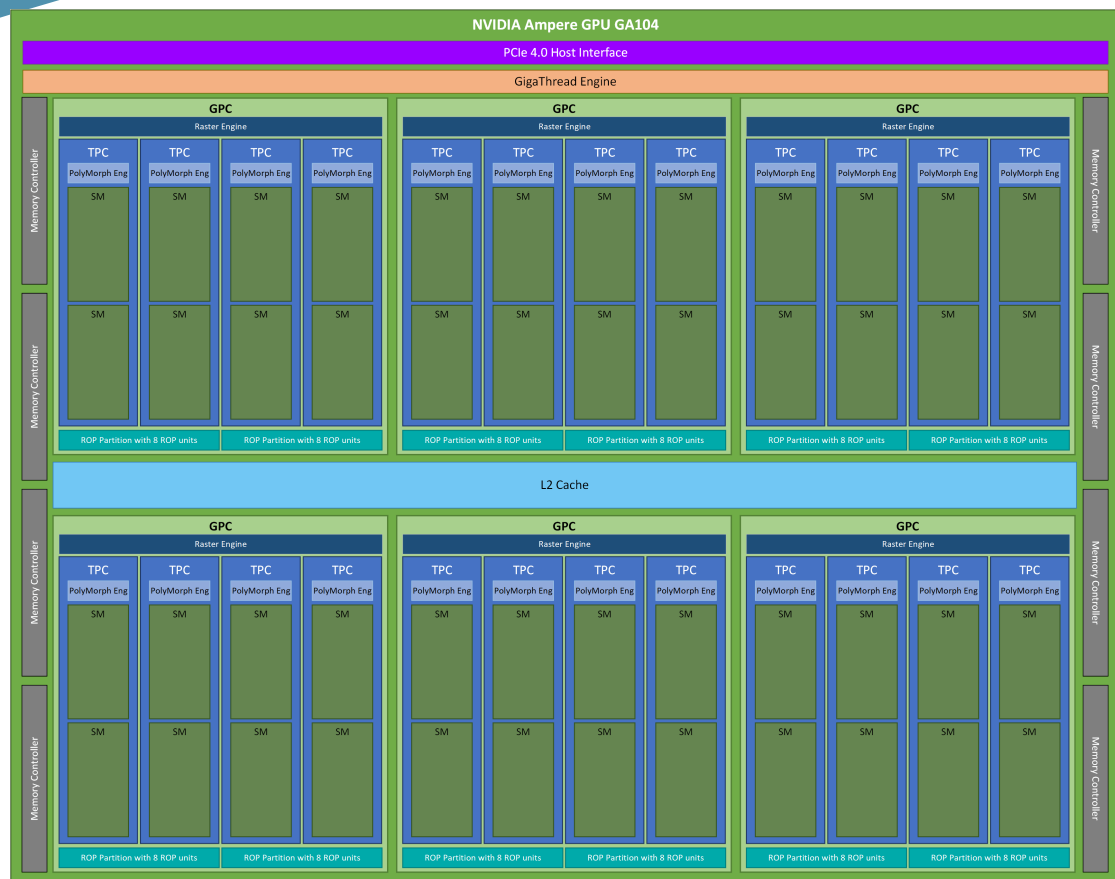
- 众核处理器比多核处理器拥有更多的特殊的处理单元
 - 它们辅助处理器进行特殊计算，被称为协处理器(Coprocessors)
- 英特尔 MIC: 多集成核心架构
 - 处理器名: 英特尔至强(Intel Xeon Phi)
 - 包含约 60 个拥有SIMD单元(向量处理单元, vector processing unit)的核心
 - 兼容x86指令集
 - 被许多排名前 500 的超级计算机使用(如天河II)
- 图形处理器[Graphics Processing Unit, GPU]
 - 英伟达(Nvidia): GTX980, Tesla K40, RTX3090, Tesla A100
 - AMD: FirePro S10000, MI100, MI200, MI300
 - 在排名前 500 的超级计算机中也很受欢迎



多核 vs. 众核

年份	处理器	核数	单精度浮点峰值计算性能	双精度浮点峰值计算性能	半精度浮点峰值计算性能	Bandwidth内存带宽	功率
2010	Intel Xeon X5650	6	0.128T	0.064T	N/A	32GB/s	95W
2013	Intel Xeon Phi 7120p	61	2.4T	1.2T	N/A	352GB/s	300W
2013	Nvidia Tesla K40	2880	4.29T	1.43T	N/A	288GB/s	235W
2012	AMD FirePro S10000 ¹	2x1792	5.91T	1.48T	N/A	480GB/s	375W
2020	Nvidia Tesla A100	6912	19.5T	9.7T	312T	1555GB/s	400W
2020	AMD Instinct™ MI100	7680	23.1T	11.5T	184.6T	1228.8GB/s	300W
2023	NVIDIA H100 PCIe 80 GB	14592	51.22T	25.61T	204.9T	2.04TB/s	350W
2023	AMD Radeon Instinct MI300	14080	47.87T	47.87T	383T	6.55TB/s	600W

什么是GPU (Graphics Processing Unit)



GPU概念

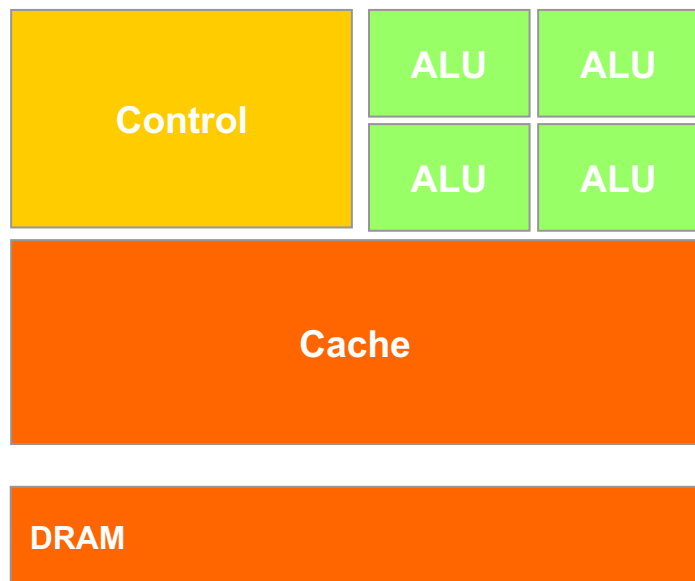
GPU即为图像处理单元，是计算机系统中的协处理器

- 早期用于它是一种专门用于处理图形和图像相关任务的处理器
- 随着计算需求的不断增加，面向通用计算，成为一种强大的**并行处理器**

CPU vs. GPU

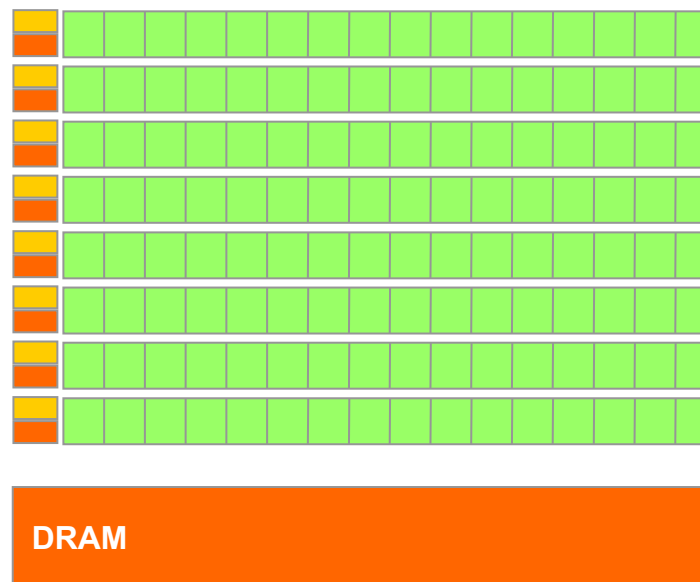


CPU



- 为**低时延的数据访问**做了很多优化
- 需要很复杂的控制逻辑
- 晶体管大部分在缓存和控制逻辑

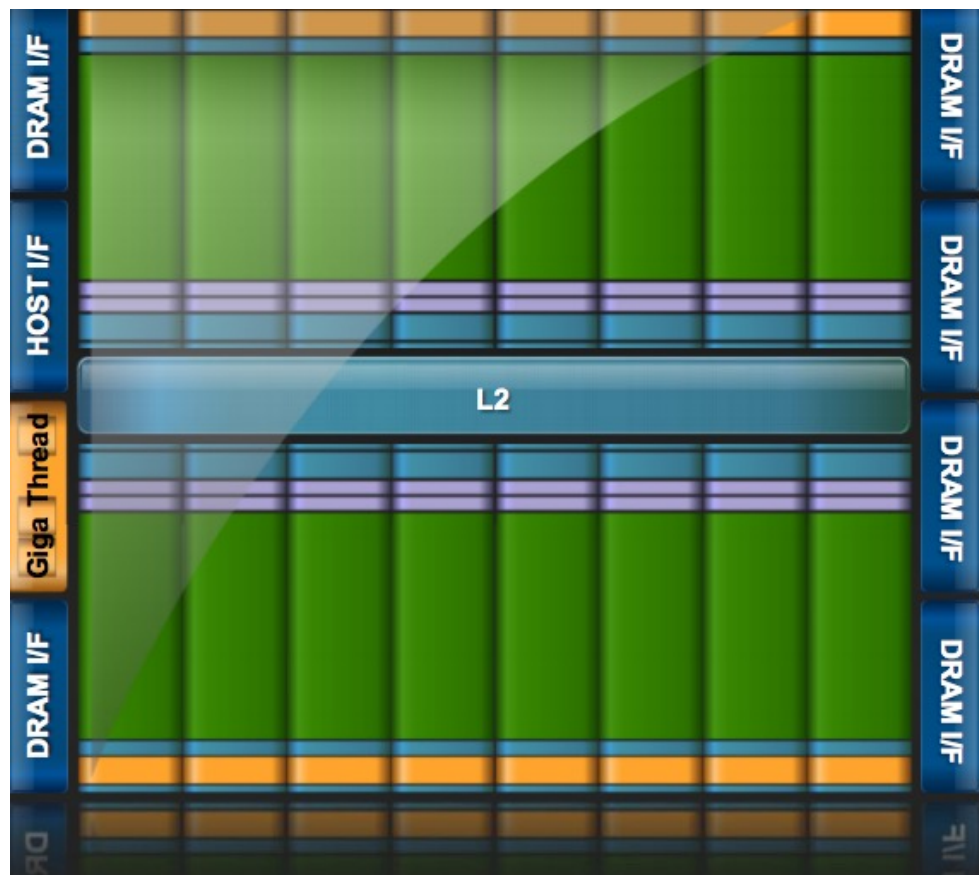
GPU



- 为**数据并行和高吞吐量**的计算优化
- 从架构上对内存访问时延有一定的容忍度
- 晶体管大部分在计算单元

GPU体系结构

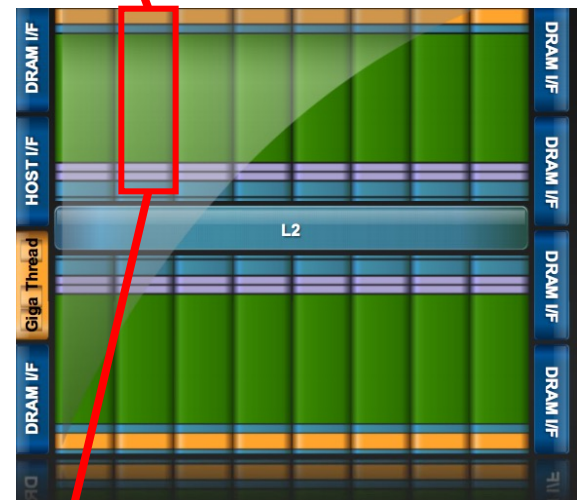
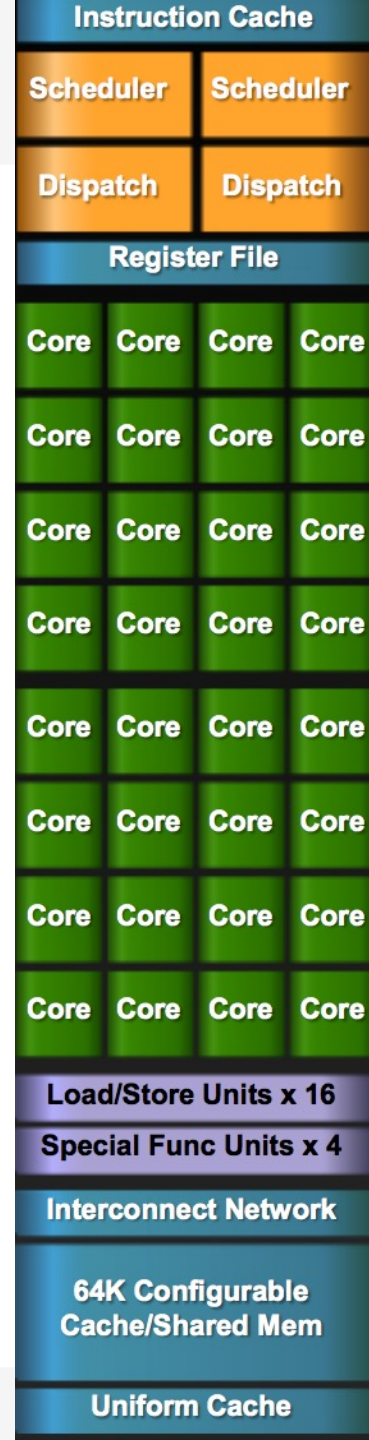
- 流多处理器 (Streaming Multiprocessors, SMs)
 - ❑ 执行实际的计算
 - ❑ 每个SM通常包含：控制单元、寄存器、缓存、执行流水线
- 显存 (Global Memory)
 - ❑ 与CPU的RAM类似
 - ❑ 可被CPU与GPU访问
 - ❑ 当前主流GPU显存大小可高达80GB
 - ❑ 带宽可达2TB/s
 - ❑ 支持ECC





GPU体系结构：流多处理器（SM） （以Fermi为例）

- 32个CUDA核
 - ❑ 每个时钟周期可执行32个FP32浮点运算、16个FP64浮点运算、32个INT32整型运算
- 2个Warp调度器
 - ❑ 最高支持1536个并发线程
- 2个加载/存储单元
 - ❑ 处理内存加载和存储
- 4个特殊功能单元
 - ❑ 加速特定类型的计算，如三角函数、指数函数、对数函数等
- ❑ 64KB可配置L1与共享内存



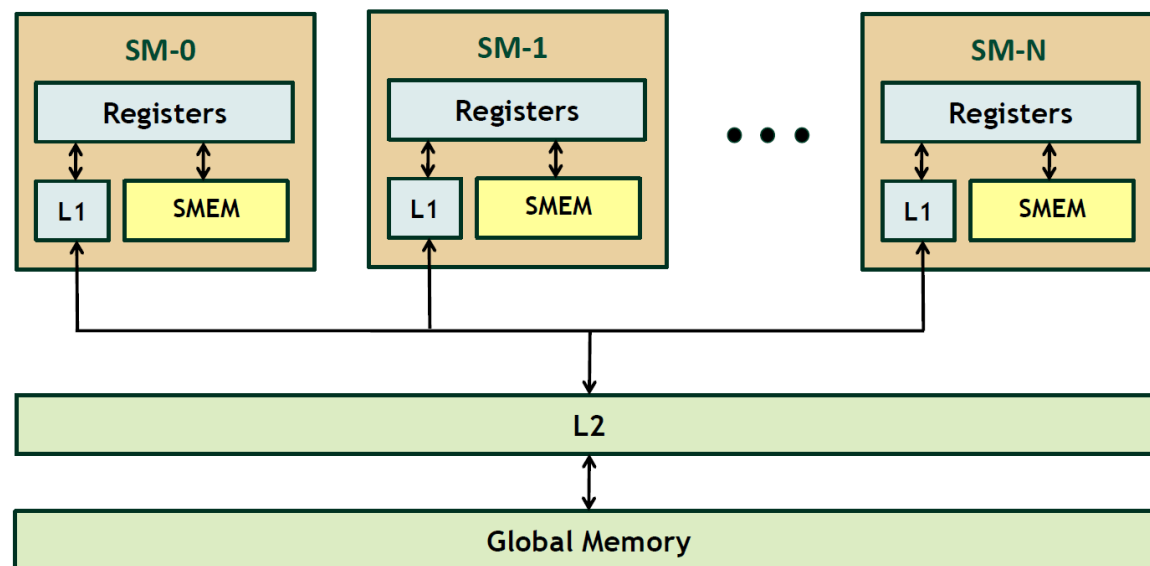
GV100

- 每个SM (计算单元)
 - FP32单元 : 64
 - FP64单元 : 32
 - int32单元 : 64
 - Tensor Core单元 : 8
- Warp调度器 : 4
- 加载/存储单元 : 32
- SFU : 16
- 每个SM (存储单元)
 - 寄存器 : 256KB
 - L1高速缓存/共享内存 : 128KB

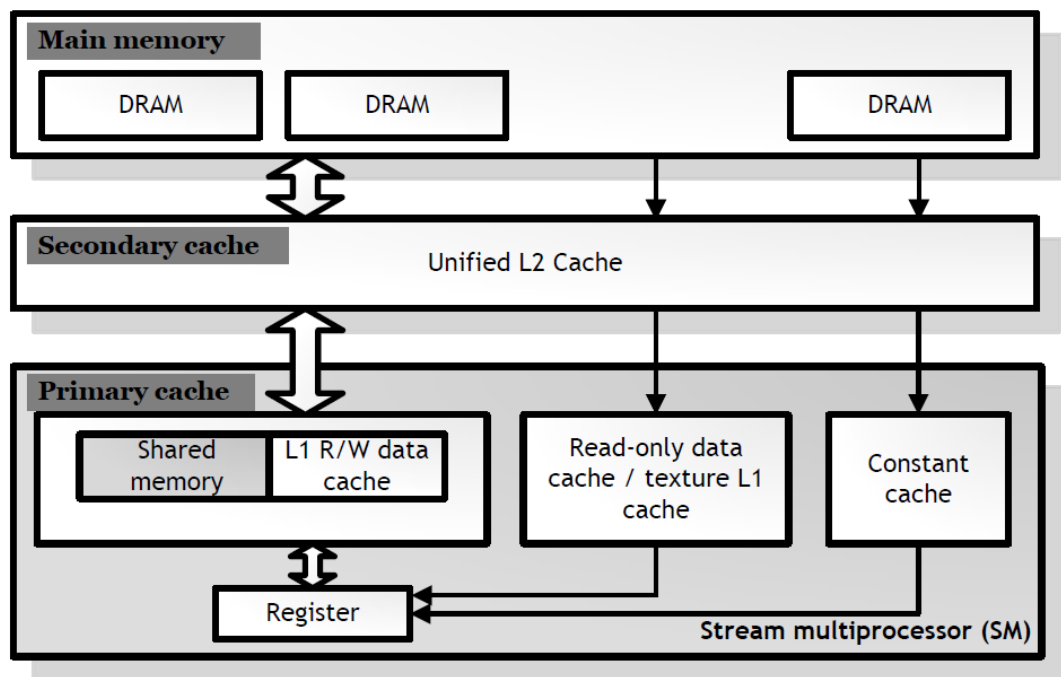


GPU体系结构：存储器层次结构

- 不同GPU架构的存储结构稍有不同
- 遵循CPU存储层次结构
 - ❑ 寄存器
 - ❑ L1高速缓存/共享内存
 - ❑ L2高速缓存
 - ❑ 主存

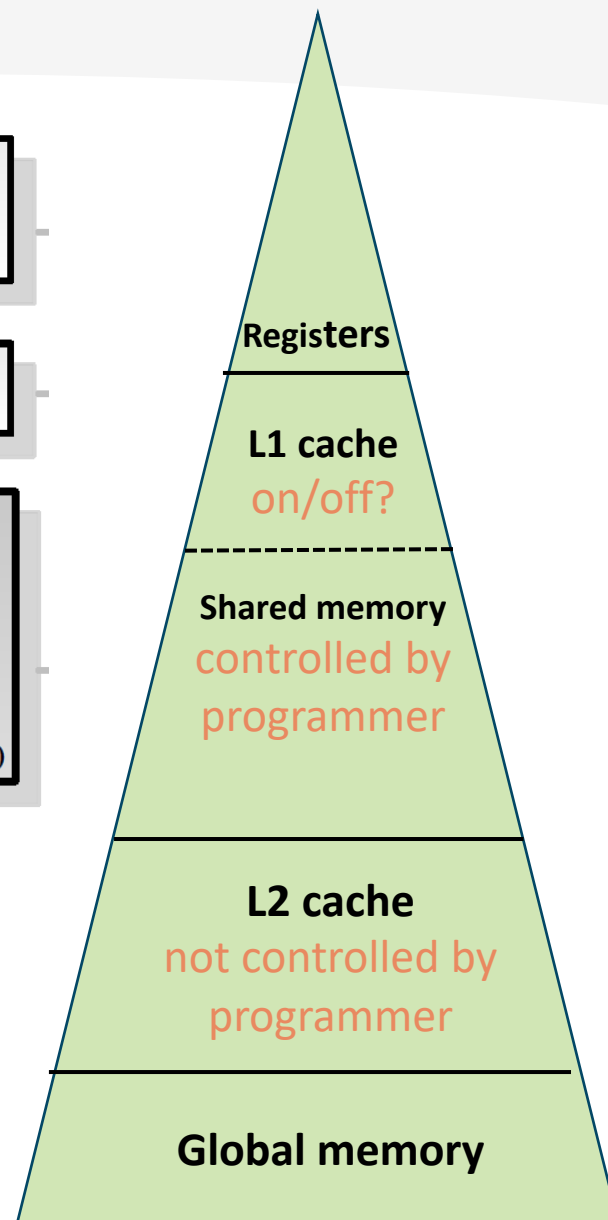


GPU体系结构：存储器层次结构



➤ 以V100为例

- ❑ 寄存器：256KB
- ❑ L1高速缓存/共享内存：96KB
- ❑ L2高速缓存：6MB（所有SM共享）
- ❑ 主存：32GB或16GB





特殊执行单元：Tensor Core

- 首次发布于Nvidia Tesla V00 GPU (2017), 1st gen Tensor Core (Volta Architecture)
 - 640个Tensor Cores (TC), 8 TCs/SM
 - **64 floating point FMA ops per TC per clock**
 - 512 FMA (or 1024 individual floating point ops) per SM
 - 40960 FMA per V100 GPU per clock

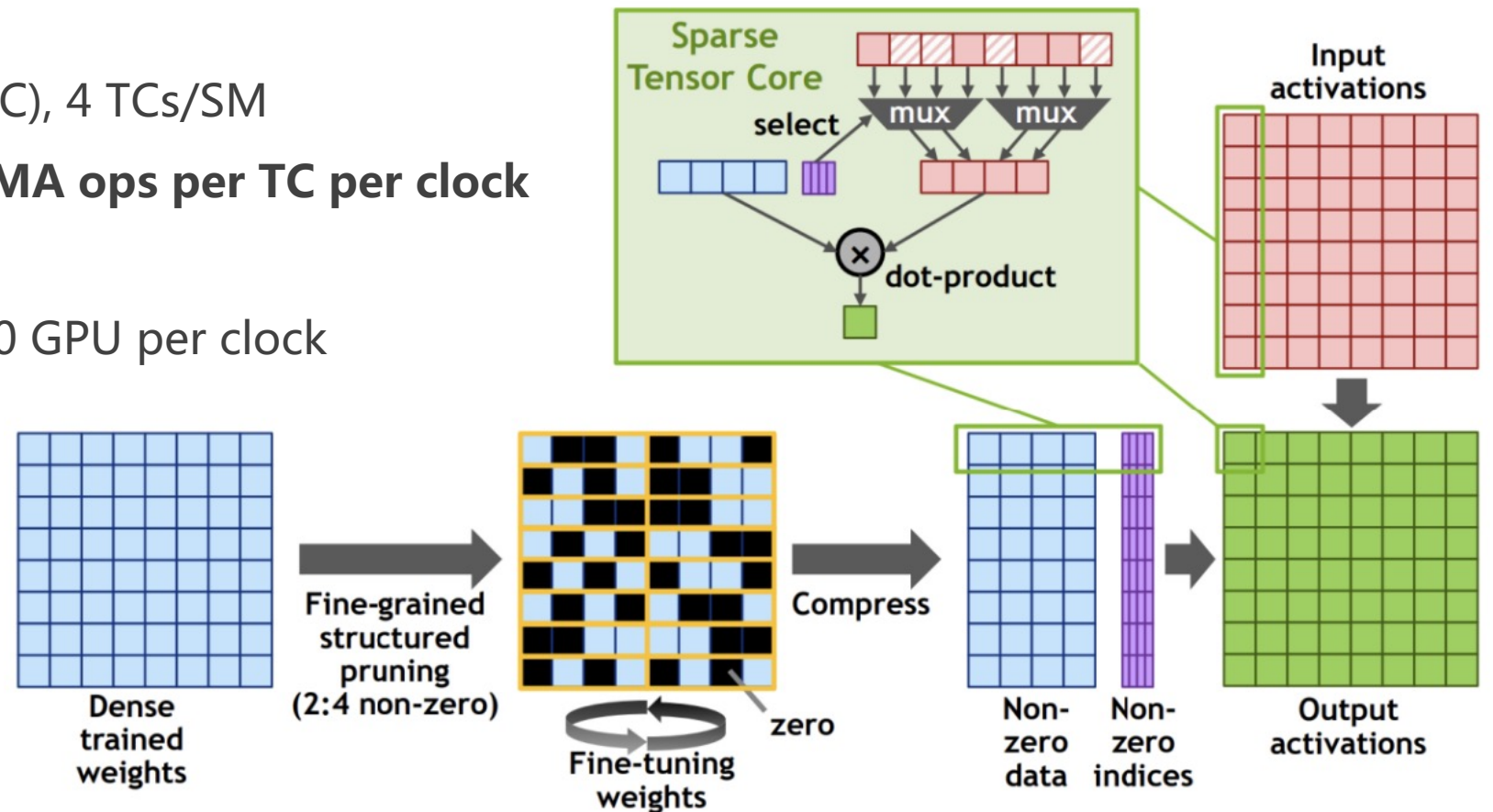
$$\mathbf{D} = \begin{pmatrix} A_{0,0} & A_{0,1} & A_{0,2} & A_{0,3} \\ A_{1,0} & A_{1,1} & A_{1,2} & A_{1,3} \\ A_{2,0} & A_{2,1} & A_{2,2} & A_{2,3} \\ A_{3,0} & A_{3,1} & A_{3,2} & A_{3,3} \end{pmatrix} \begin{pmatrix} B_{0,0} & B_{0,1} & B_{0,2} & B_{0,3} \\ B_{1,0} & B_{1,1} & B_{1,2} & B_{1,3} \\ B_{2,0} & B_{2,1} & B_{2,2} & B_{2,3} \\ B_{3,0} & B_{3,1} & B_{3,2} & B_{3,3} \end{pmatrix} + \begin{pmatrix} C_{0,0} & C_{0,1} & C_{0,2} & C_{0,3} \\ C_{1,0} & C_{1,1} & C_{1,2} & C_{1,3} \\ C_{2,0} & C_{2,1} & C_{2,2} & C_{2,3} \\ C_{3,0} & C_{3,1} & C_{3,2} & C_{3,3} \end{pmatrix}$$

FP16 or FP32 FP16 FP16 FP16 or FP32

4x4x4

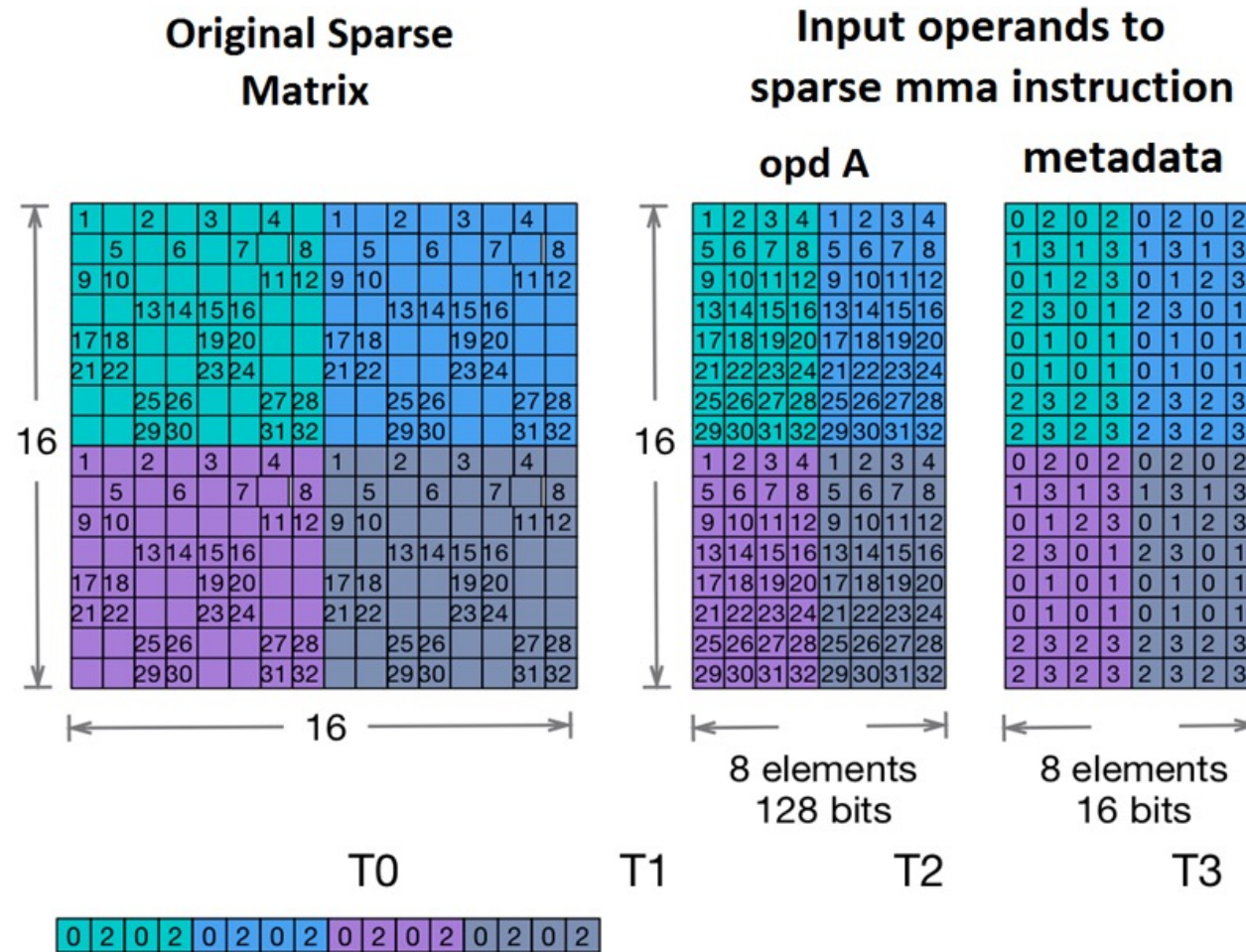
特殊执行单元：Sparse Tensor Core

- 首次发布于Nvidia Tesla A100 GPU (2020), 3rd gen Tensor Core (Ampere Architecture)
 - 128 SMs
 - 512个Tensor Cores (TC), 4 TCs/SM
 - **256 floating point FMA ops per TC per clock**
 - 1024 FMA per SM
 - 131072 FMA per A100 GPU per clock





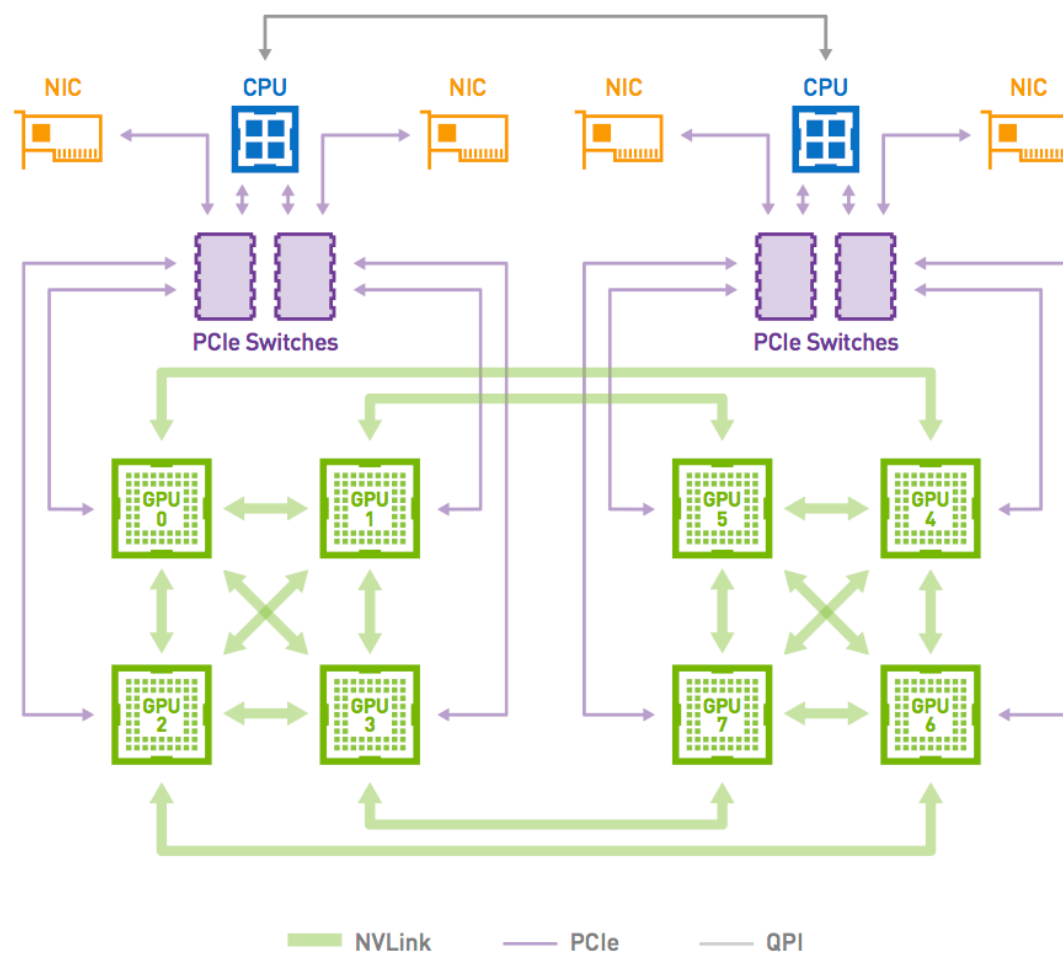
特殊执行单元：Sparse Tensor Core



sparsity selector = 0
indicates that
thread T0 contains
metadata.

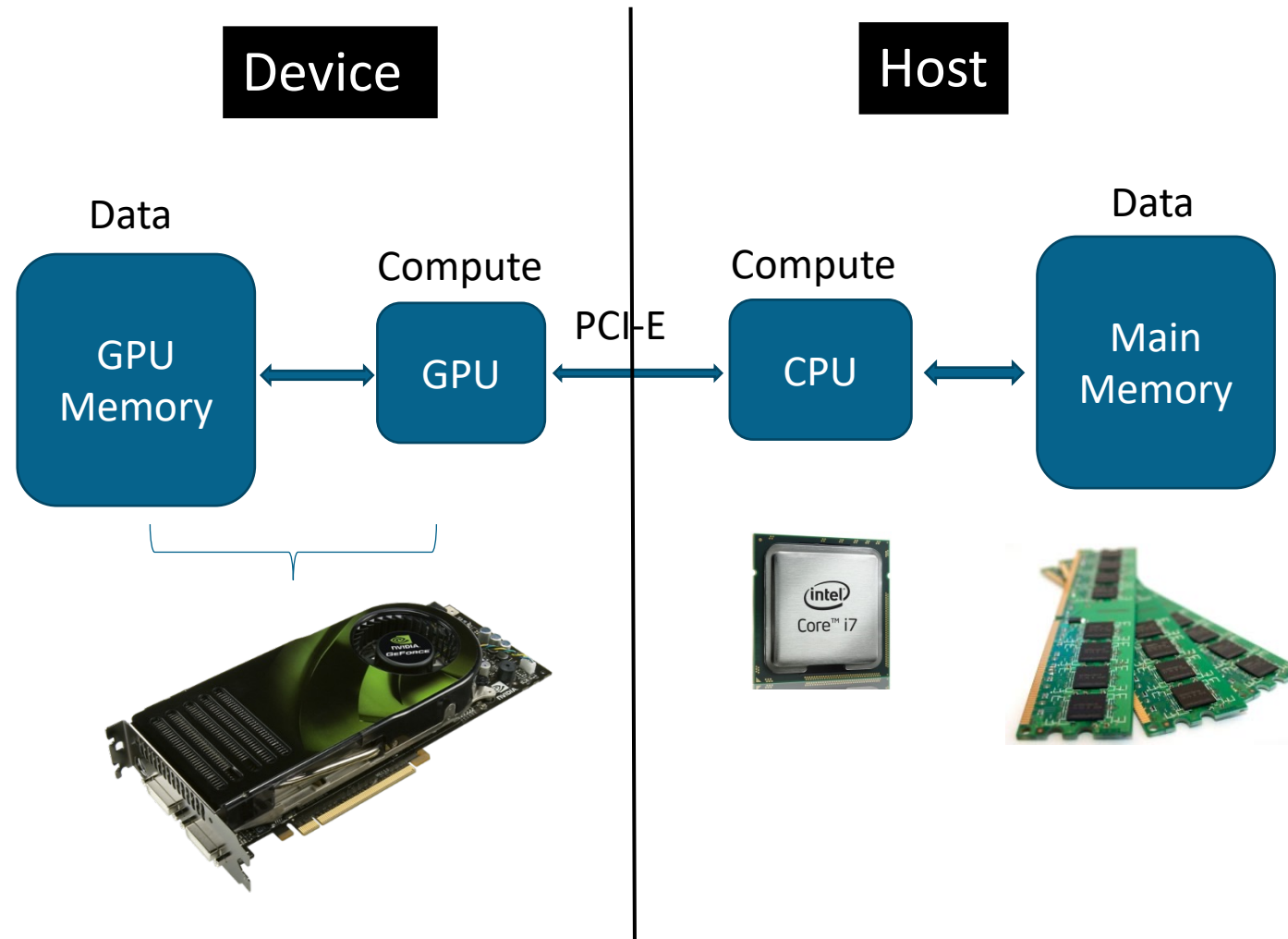
GPU互连：NVLink

- 首次发布于Nvidia Tesla P100 GPU (2016) (Pascal Architecture)的DGX-1系统中 (8张P100)
 - 每个P100有4个NVLink连接点，每个连接点提供最高20GB/s点到点的传输带宽
- 最高80GB/s
 - 每个GPU同时与其相连的GPU传输数据





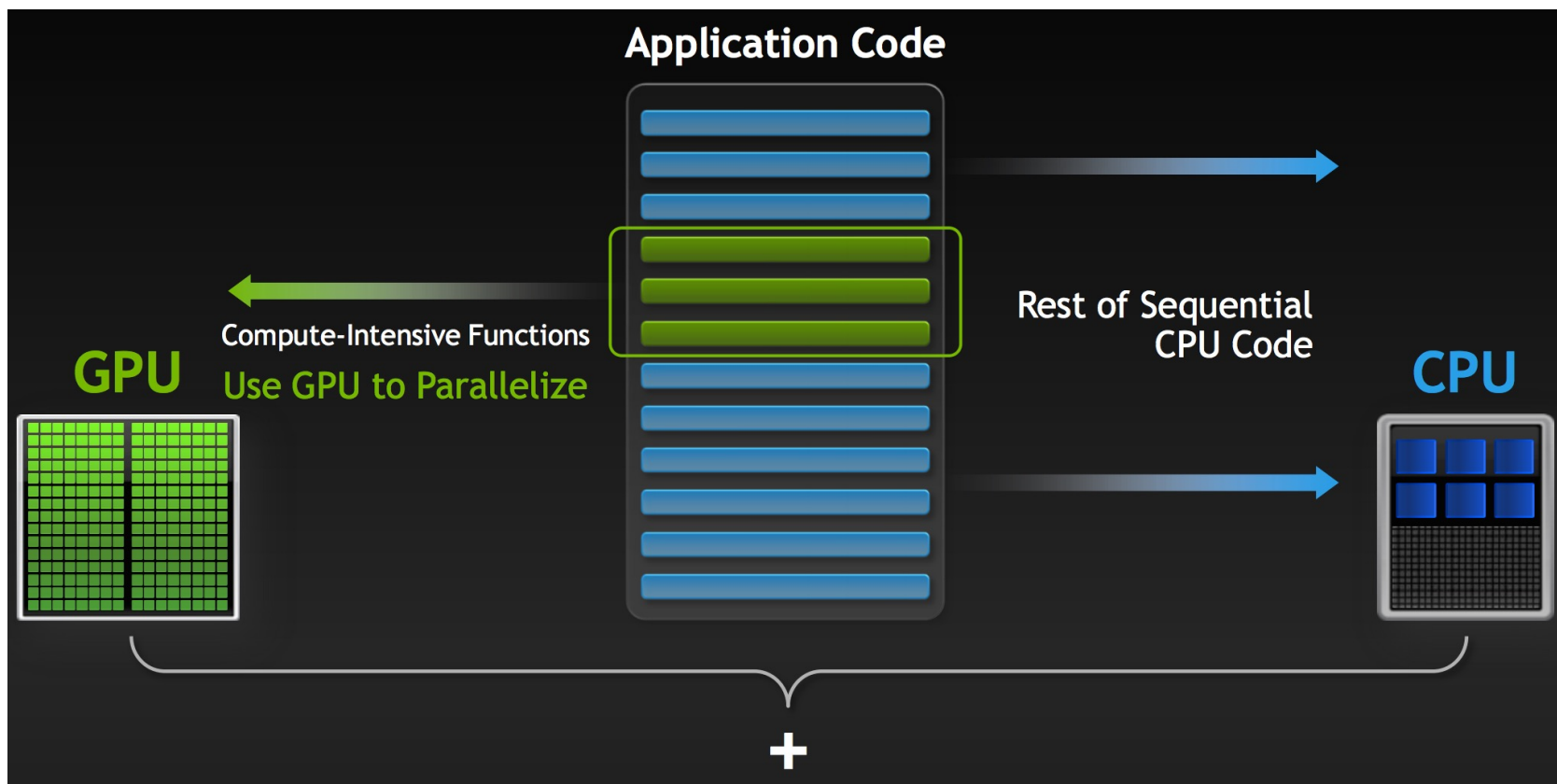
GPU计算：一种异构计算模式



GPU计算



- 把计算任务Offload到GPU中





目录 | CONTENT

- 1 GPU架构与编程模型
- 2 **CUDA编程**
- 3 GPU访存优化
- 4 GPU控制流和指令优化



CUDA编程：CUDA历史

- CUDA：Compute Unified Device Architecture
 - ❑ 做为Nvidia GPU通用计算解决方案
- 2007年：CUDA1.0
 - ❑ 支持C/Fortran语言，CUDA做为语法/接口扩展以及编译器
- 2008年：CUDA2.0
 - ❑ 支持Runtime API和Driver API
-
- 2011年：CUDA4.0
 - ❑ CUDA Stream/Event API
-
- 2014年：CUDA6.0
 - ❑ Unified memory
- 2017年：CUDA9.0
 - ❑ MPS (多进程管理)，Tensor Core APIs
- 2018年：CUDA10.0
 - ❑ CUDA Graphs
- 2020年：CUDA11.0
 - ❑ 支持Arm平台(arm64 SBSA)，MIG
- 2022/23年：CUDA12.0/13.0
 - ❑ 可编程L2缓存



CUDA设备和线程

- 在CUDA中，一个计算设备（ Device ）是指
 - ❑ 相对于CPU的协处理器（ 如GPU ） ， CPU端则称为Host
 - ❑ 有独立于CPU内存的内存（ 如GPU显存 ， Device memory ）
 - ❑ 可以并行执行多个线程（ 上百或上千 ）
 - ❑ 通常是指GPU ， 但也可以是其他并行处理器
- 在计算设备上执行的程序表达为： device kernel （ 或简称为kernel ）
 - ❑ 一个kernel可以并行执行很多线程
- 与CPU线程的区别
 - ❑ CUDA线程是非常轻量级的：创建开销很低
 - ❑ 通常需要上千个线程并行执行才能发挥GPU的计算性能



CUDA计算性能 (Compute Capability, CC)

- Compute capability (CC) 用于标识在每一类GPU硬件的某些特殊特性
- CC有一个大版本号和一个版本号组成 (x.y形式)
 - ❑ 如 : 1.3 , 2.0 , ... , 7.0 , 9.0 (H100 GPU)
 - ❑ 同样大版本号的设备具有同样的核心架构
 - ❑ 小版本号用于区别小的改进

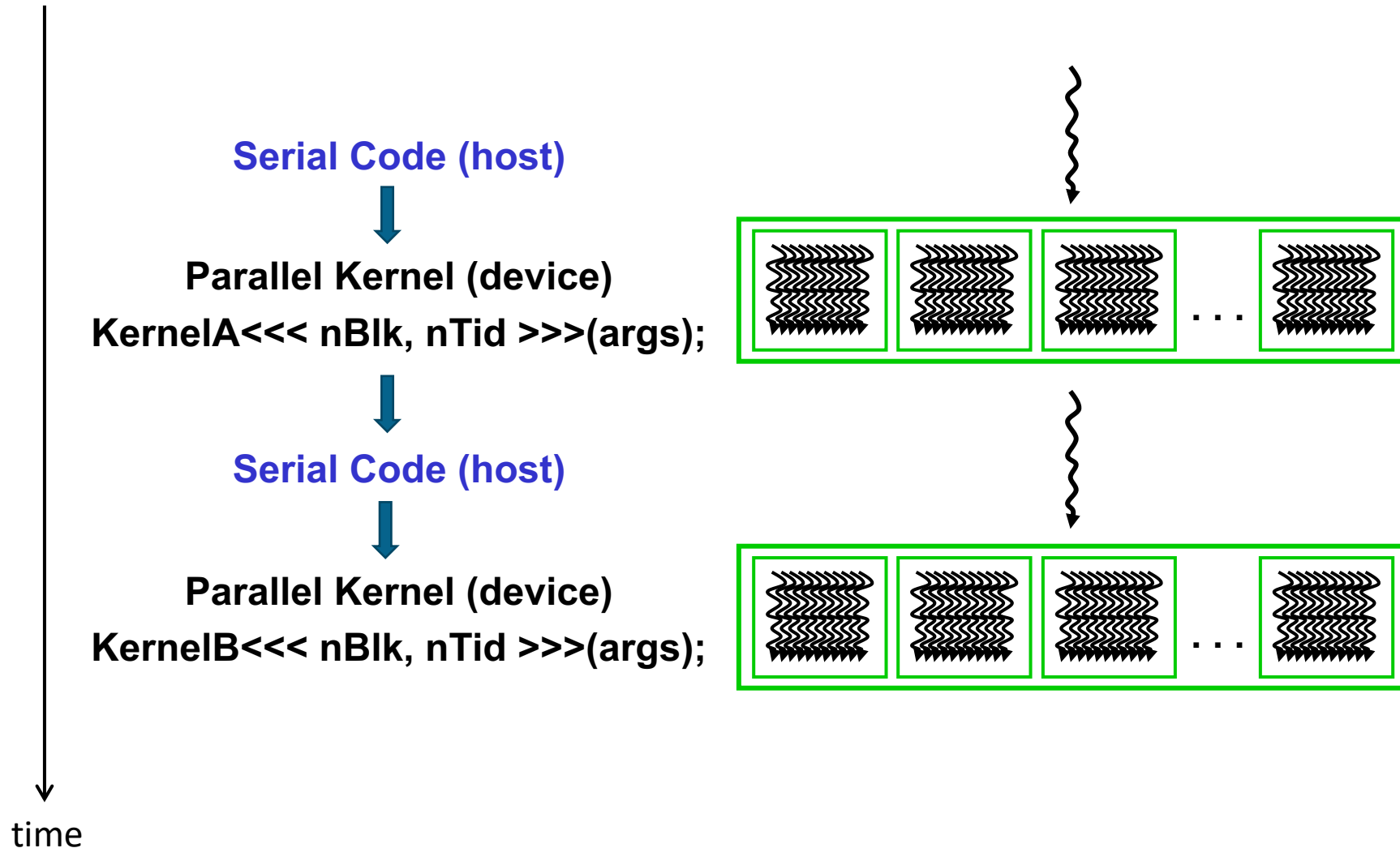


CUDA C/C++

- CUDA支持多种编程语言
 - **C/C++**、Fortran、Java、C#、Python
- CUDA C/C++是C/C++的扩展
 - 一系列新的关键字
 - 一系列API函数
 - 一系列高效库：如cuBLAS、cuFFT、cuDNN
- CUDA C/C++：编写Host+device的混合程序
 - 串行或CPU并行代码在host端执行
 - 高并行代码在device端执行，也称为kernel函数



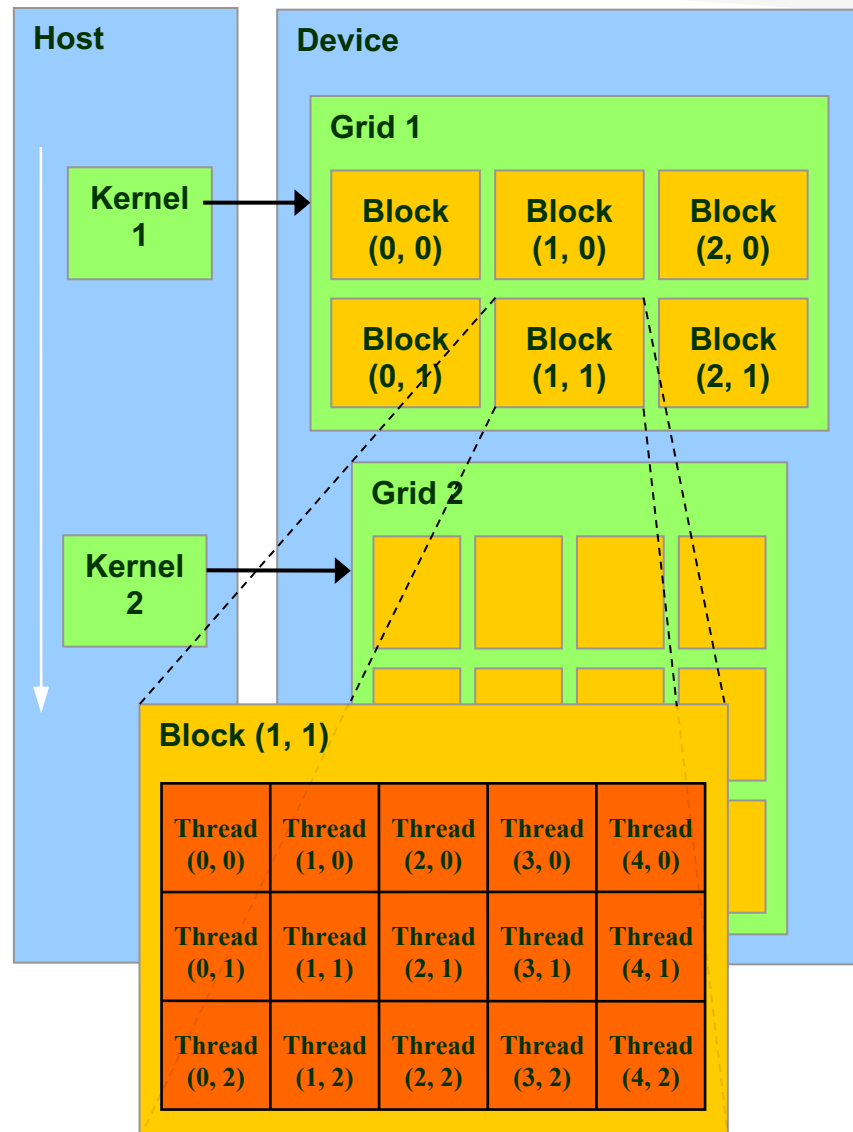
CUDA C/C++ 程序的执行流程



线程

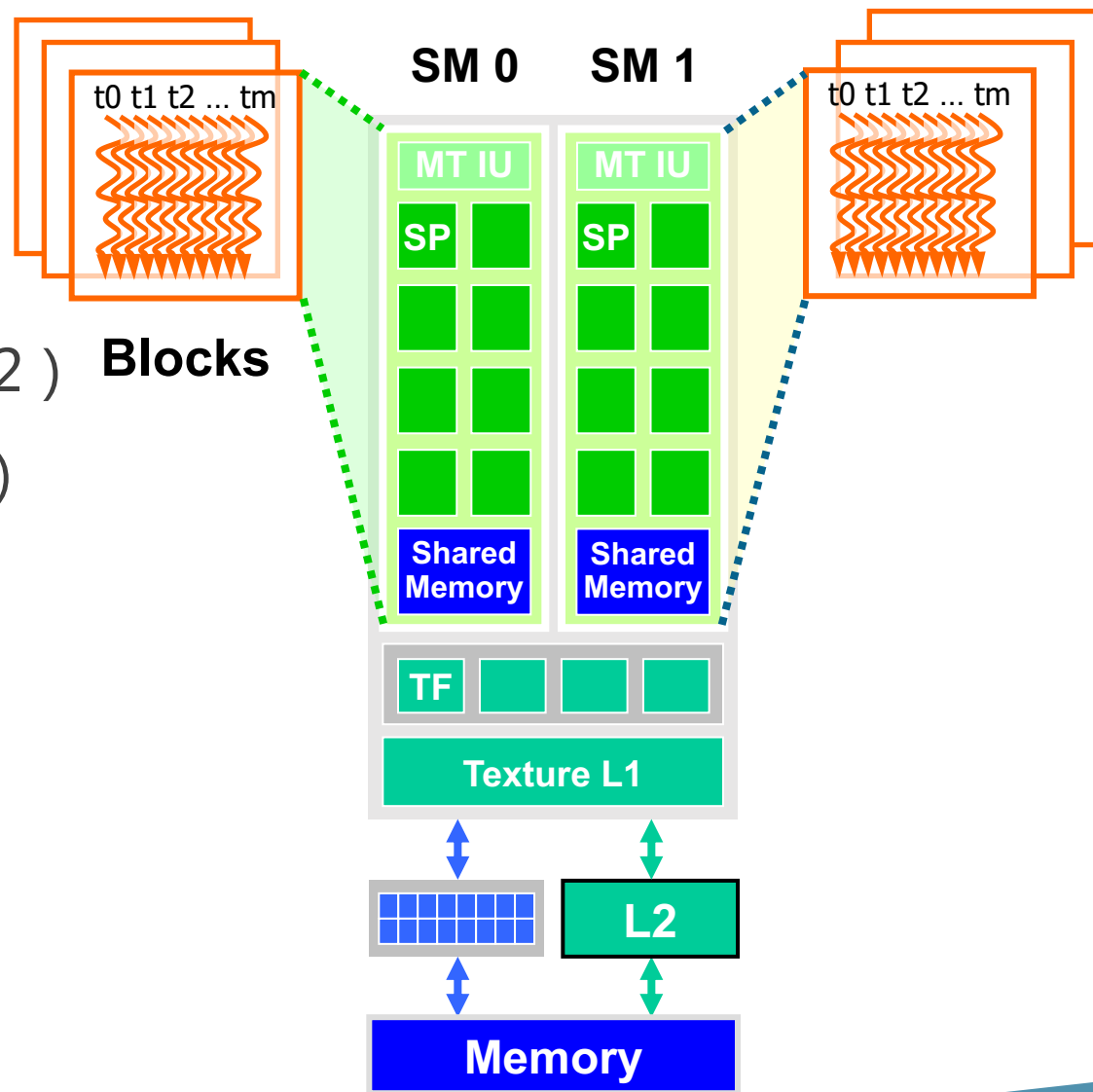


- 以线程来执行指令
- 32个线程组成一个Warp
 - ❑ 最小的调度单位
- Block：多个线程组成一个Block
 - ❑ 多个Block组成一个Grid
 - ❑ 不同Block按ID顺序分给不同的SM执行
 - ❑ 当SM执行完Block里所有线程的任务，即会调入其他Block进行执行



线程块 (Thread Block)

- 以Block为单位线程分到SM
 - ❑ 每个SM同时允许有限个Blocks (如32)
 - ❑ 每个SM的线程数也有上限 (如2048)
- 线程在不同的SM并发执行
 - ❑ SM维护线程ID
 - ❑ SM里的Warp Scheduler负责调度线程





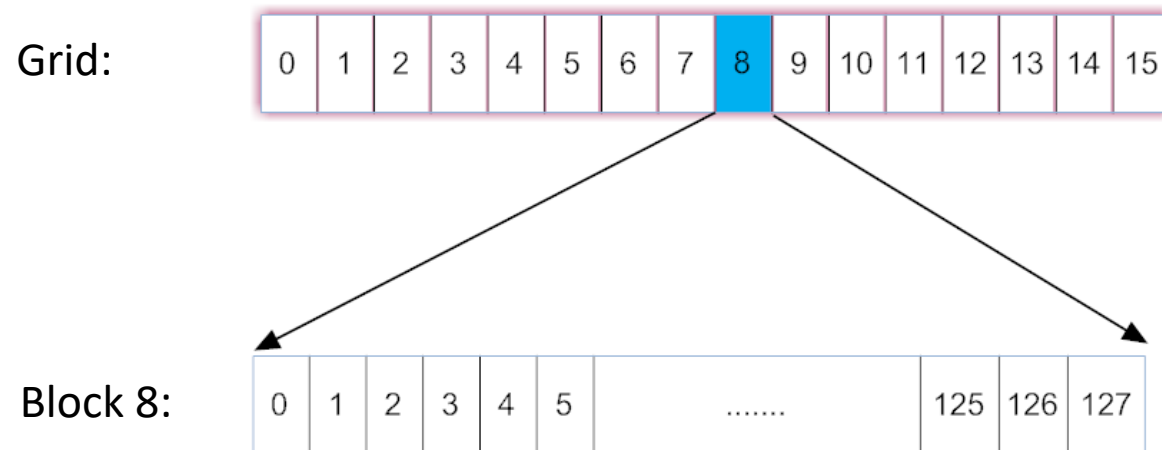
CUDA线程组织

- 多个线程块 (Block) 可以组成1维、2维或3维的Grid , 在加载时指定
 - ❑ 使用gridDim内建变量获取指定的维度
 - `gridDim.x`, `gridDim.y`, `gridDim.z`
 - `x`, `y`, `z`的维度按不同的CC有不同的限定 , 如CC>3.0 , `x`小于 2^{31} , `y,z`小于65536
 - ❑ 使用blockDim和 blockDim内建变量获取block维度和索引
 - `blockDim.x`, `blockDim.y`, `blockDim.z`
 - `blockIdx.x`, `blockIdx.y`, `blockIdx.z`
- 多个线程组成1维、2维或3维的Block , 在加载时指定
 - ❑ 通过内建变量threadIdx获取线程索引
 - `threadIdx.x`, `threadIdx.y`, `threadIdx.z`



举例：1维组织

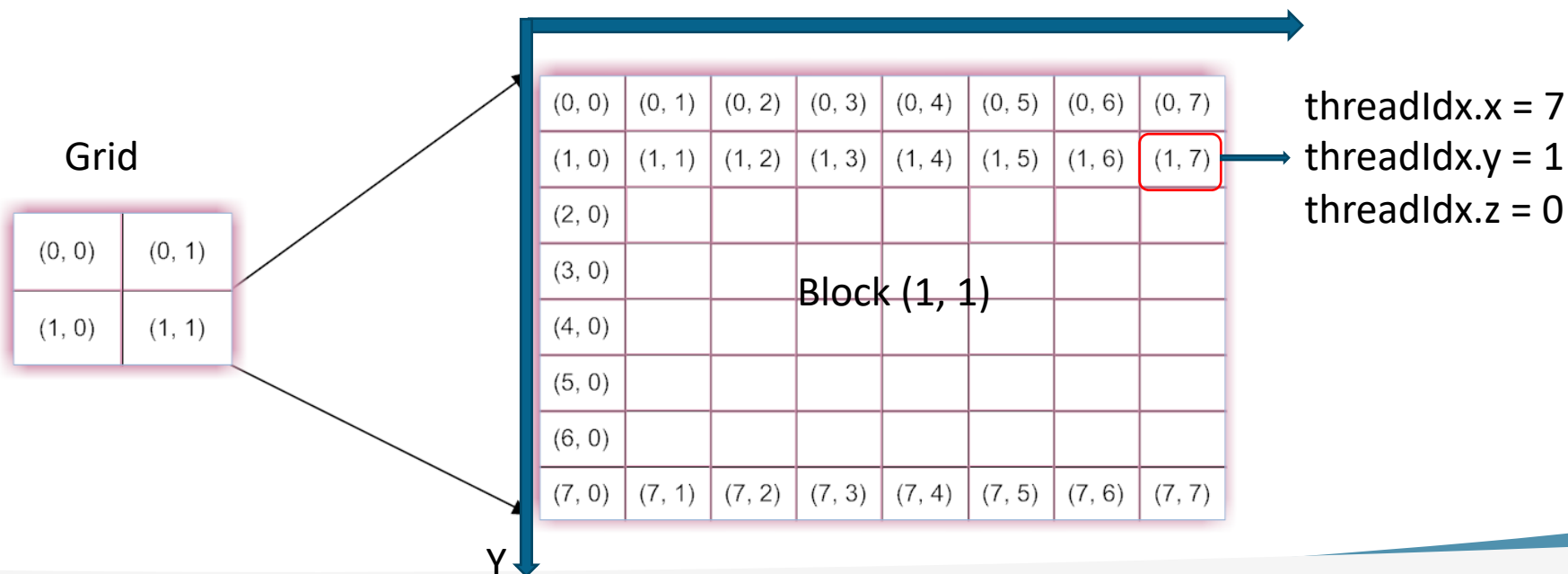
- 假设配置1个1维的grid包含16个blocks，每个block包含1维的128个线程
 - ❑ `dim3 dimBlock(128, 1, 1); // 把y和z维度设为1`
 - ❑ `dim3 dimGrid(16, 1, 1); // 把y和z维度设为1`
 - ❑ `gpuKernel<<<dimGrid, dimBlock>>>(...);`





举例：2维组织

- 假设配置1个2维的grid包含 $2 \times 2 = 4$ 个blocks，每个block包含2维的 $8 \times 8 = 64$ 个线程
 - ❑ `dim3 dimBlock(8, 8, 1); // 把z维度设为1`
 - ❑ `dim3 dimGrid(2, 2, 1); // 把z维度设为1`
 - ❑ `gpuKernel<<<dimGrid, dimBlock>>>(...);`





GPU硬件资源分配

- 一旦Kernel被加载，CUDA则按指定配置生成线程（以block和grid组织）
- GPU硬件资源（即SM）的分配则按每个block为单位的
 - ❑ 假设有N个blocks，GPU有P个SM，且 $P < N$
 - ❑ 则平均每个SM需要执行 N/P 个blocks
- ❑ 活跃blocks
 - ❑ 每个SM可以同时执行多个blocks（有限个），我们称为活跃blocks（active blocks）
 - ❑ 当前的active blocks执行完后，再分配新的blocks成为active blocks执行

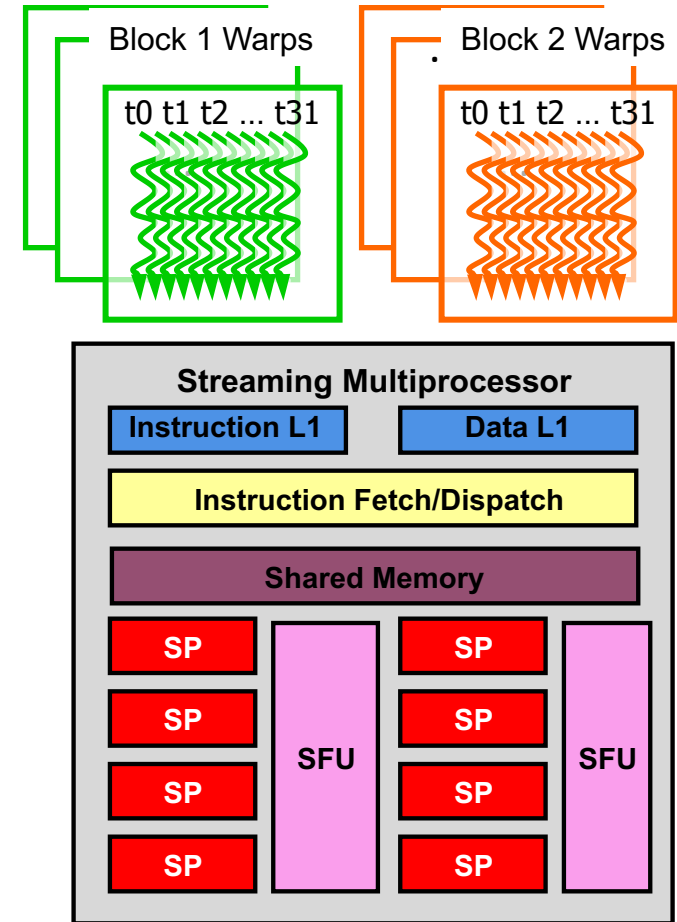


线程调度

- 一旦一个block被分配到一个SM中，则这个block进行划分为多个warps
 - ❑ 每个warp有32个线程
 - ❑ Warp 0: threads 0 – 31
 - ❑ Warp 1: threads 32 – 63
 - ❑ Warp 2: threads 64 - 95
 - ❑
- Warp为GPU线程调度的最小单位
 - ❑ 一个warp里的32线程按SIMD模式执行
 - ❑ 即32个线程执行相同的指令

线程调度

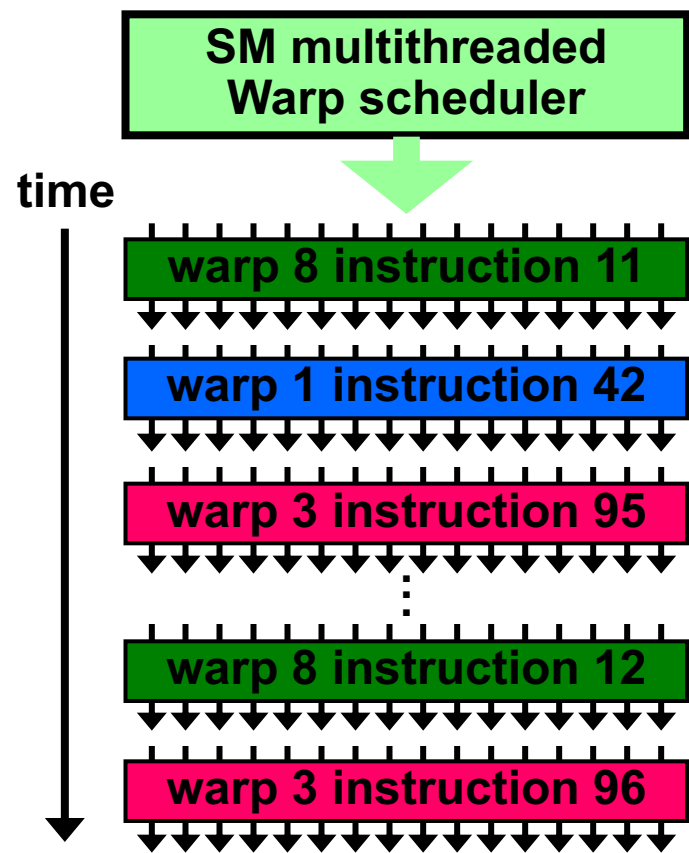
- 每个线程块被划分为多个包含32个线程的Warp
 - 这是实现时的决策，并非 CUDA 编程模型的一部分
- Warp是 SM 中的调度单位
- 如果某个SM被分配了3个线程块，每个线程块有512个线程，那么调度时共有多少个Warp呢？
 - 每个线程块会被划分为 $512/32 = 16$ Warps
 - 共有 $16 * 3 = 48$ Warps
 - 在任何时间点，48 个 Warps 中的某些会被选择进行取指和执行



Warp调度



- SM从硬件的Warp调度器实现0开销的Warp调度
 - 操作数已经Ready的执行即可被调度执行
 - 可被调度Warp按优先级选择执行
 - 一个Warp里的所有线程（32）执行同条指令
- 举例：G80架构，每条执行分发需要4个Cycles
 - ❑ 如果每4条执行需取一次Global memory
 - ❑ 最少需要13个Warps同时执行来容忍200-cycle的访存延时





Blocks/warps/threads的硬件限制

- 由于硬件资源的限制，如SM数量、寄存器数量、warp调度器数量、共享内存大小等，活跃blocks/warps/threads的数量也有限制，如下表

Compute Capability (CC)	1.x	2.x	3.x	5.0	7.0
Max. number of active blocks per SM	8	8	16	32	32
Max. number of active warps per SM	24 → 32	48	64	64	64
Max. number of active threads per SM	768 → 1024	1536	2048	2048	2048
Max. number of threads per block	512	1024	1024	1024	1024



Blocks/warps/threads的硬件限制

- 寄存器和共享内存速度快但大小有限
- 如果单个线程使用大量寄存器和/或共享内存，“活动线程”（活跃Block的线程）的数量会减少，从而可能导致性能低下
- 例如，在计算能力为2.x的GPU中，每个 SM 有 32768 个寄存器，最多可支持 1536 个活动线程
 - 如果单个线程使用 21 个寄存器，则最多可有1536 个活动线程（ $1536 \times 21 < 32768$ ）
 - 如果单个线程使用 22 个寄存器，那么最多只能有1489个线程。如果线程块大小为 512，那么只能有2个活动线程块，即 1024 个活动线程



Blocks/warps/threads的硬件限制-举例

➤ 假设在CC=3.0的设备中，下列情况Active threads最多是多少？

- ❑ Case 1: block size=8x8=64
- ❑ Case 2: block size=16x16=256
- ❑ Case 3: block size is 32x 32=1024
- ❑ Case 4: block size is 64x 64=4096



Compute Capability (CC)	1.x	2.x	3.x	5.0	7.0
Max. number of active blocks per SM	8	8	16	32	32
Max. number of active warps per SM	24 → 32	48	64	64	64
Max. number of active threads per SM	768 → 1024	1536	2048	2048	2048
Max. number of threads per block	512	1024	1024	1024	1024



CUDA内存管理

- 在CUDA中，host（即CPU端）和device（即GPU端）有独立的内存地址空间
- 在device上执行一个kernel，需要
 - 申请设备内存
 - 将host内存上数据传输到device上内存
- kernel执行完毕后，将device内存的结果传回host内存



CPU与GPU内存拷贝

- Host memory与Global/Texture/Constant memory间进行拷贝
 - ❑ Global memory较为常见
 - ❑ Texture和Constant memory有特别用法
- GPU内存 (Global) 操纵函数
 - ❑ 物理内存开辟在Global memory中
 - ❑ 需要从CPU主存从拷贝到Global memory方被GPU线程使用
 - ❑ Global memory数据需要拷贝到CPU主存才可被主进程使用

cudaMemcpyKind

cudaMemcpyHostToHost

cudaMemcpyHostToDevice

cudaMemcpyDeviceToHost

cudaMemcpyDeviceToDevice

```
cudaError_t cudaMalloc(void** devPtr, size_t size)
```

```
cudaError_t cudaMemcpy(void* dst, const void* src, size_t count,  
    enum cudaMemcpyKind kind, cudaStream_t stream = 0)
```

```
cudaError_t cudaMemset(void* devPtr, int value, size_t count)
```

```
cudaError_t cudaFree(void* devPtr)
```

Standard C Functions

malloc

memcpy

memset

free

CUDA C Functions

cudaMalloc

cudaMemcpy

cudaMemset

cudaFree



Kernel函数

- 在CUDA中，kernel函数指示在GPU上执行
 - ❑ 所有的线程都执行同样的kernel代码
- 用__global__关键字进行定义
- __device__和__host__可以同时使用
 - ❑ 编译器会产生两个版本

	Executed on the:	Only callable from the:
<code>__device__ float DeviceFunc()</code>	device	device
<code>__global__ void KernelFunc()</code>	device	host
<code>__host__ float HostFunc()</code>	host	host



加载Kernel函数

- 加载Kernel函数需要指定的配置
 - Grid数量与维度
 - Block数量与维度
 - 共享内存大小
 - <<<>>>包含住

```
__global__ void KernelFunc(...);  
dim3    DimGrid(100, 50);    // 5000 thread blocks  
dim3    DimBlock(4, 8, 8);   // 256 threads per block  
size_t  SharedMemBytes = 64; // 64 bytes of shared memory  
KernelFunc<<< DimGrid, DimBlock, SharedMemBytes >>>(...);
```



加载Kernel函数：异步执行

- 加载Kernel函数与Host代码是异步的
 - 即Kernel函数调用之后，CPU将继续执行后续代码
 - 遇到同步API时，即CPU端需要等待Kernel完成，如cudaMemcpy()
 - 专用的同步API：cudaDeviceSynchronize()

cudaError_t cudaDeviceSynchronize(void);



Kernel函数的一些限制

- Kernel内的线程只能访问device内存
- 返回类型必须为void
- 不支持不定长参数
- 不支持静态变量
- 不支持函数指针



举例：基于CUDA的向量加法

```
#include <cuda.h>

void vecAdd(float *A, float *B, float *C, int n)
{
    int size = n * sizeof(float);
    float  *A_d, *B_d, *C_d;

    //step 1: allocate device memory for A, B, and C; then copy A and B
    to device memory

    //step 2: launch the kernel code to perform the actual vector
    addition on GPU

    //step 3: copy C from device memory and free device memory
}
```



举例：基于CUDA的向量加法

```
int main ()
{
    int n = 10000;
    // allocate and initialize host (CPU)
    memory
    float *H_A = ..., *H_B = ..., *H_C = ...;
    // allocate device (GPU) memory
    float *A_d, *B_d, *C_d;
    cudaMalloc(...); ...
    // copy host memory to device
    cudaMemcpy(...); ...
    // run 16 blocks of 256 threads each
    vecAdd<<< ceil(n/256.0), 256 >>>(d_A,
    d_B, d_C, n);
    // copy result to host
    cudaMemcpy(...);
    cudaFree(A_d); ...
}
```

```
// compute vector sum C = A + B
// each thread performs one pair-wise addition
__global__ void vecAdd( float *A, float *B,
float *C, int n)
{
    // locate the memory
    int i = threadIdx.x + blockDim.x *
    blockIdx.x;
    // perform the addition
    if(i < n) C[i] = A[i] + B[i];
}
```



异常处理

➤ 定义异常处理宏

```
#define CHECK(call)
{
    const cudaError_t error = call;
    if (error != cudaSuccess)
    {
        printf("Error: %s:%d, ", __FILE__, __LINE__);
        printf("code:%d, reason: %s\n", error, cudaGetErrorString(error));
        exit(1);
    }
}
```

E.g.:

```
CHECK(cudaMemcpy(...));
```



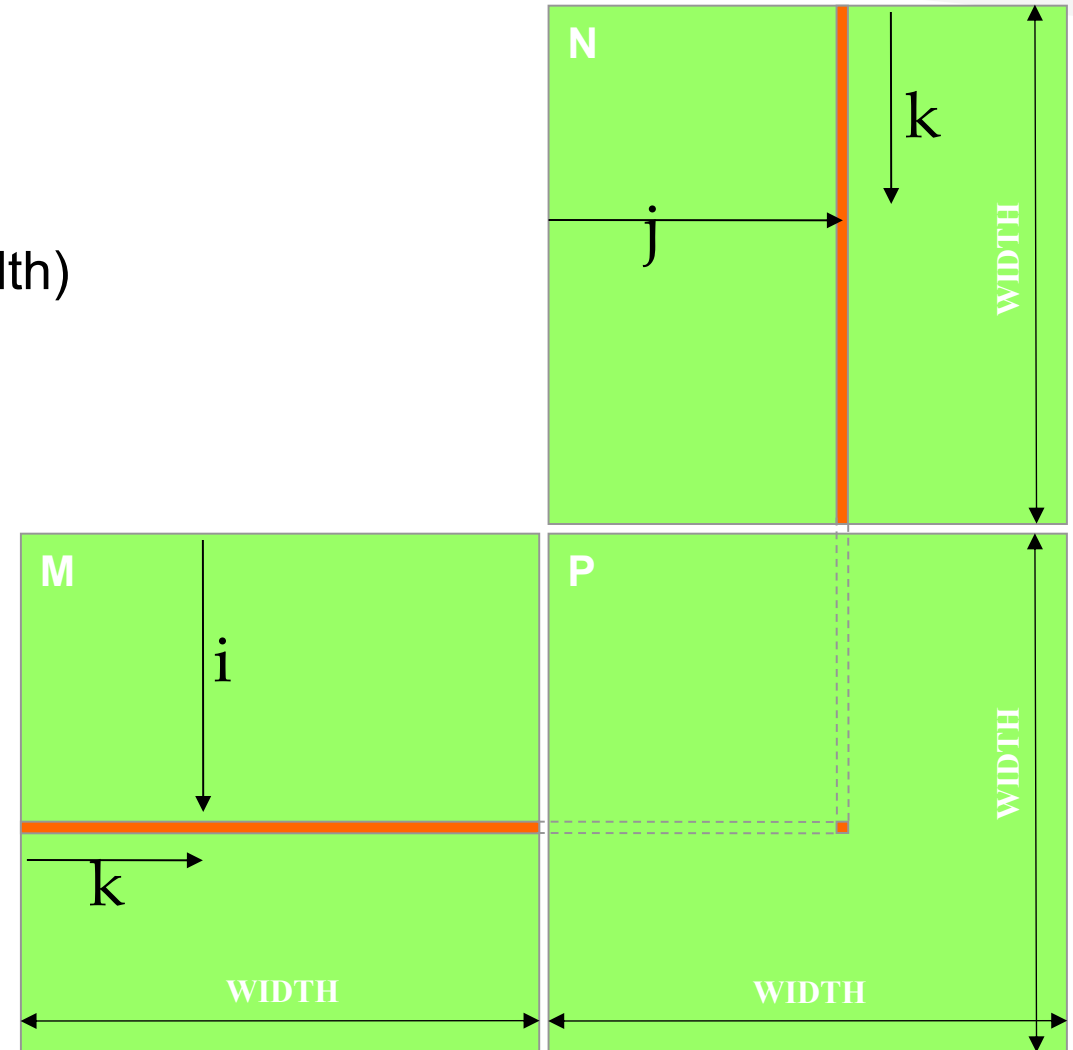

案例: 矩阵乘法

- 问题描述: $P = M \times N$
- M, N, P 都是正方形矩阵: $WIDTH \times WIDTH$
- GPU并行实现
 - 最直接的想法: P 中的每一个元素的计算用一个GPU线程来完成
 - 计算 P 的一个元素: 向量点乘
 - P 中每个元素的计算都是独立互斥的
 - Question: 如何规划GPU的线程块?



案例: 矩阵乘法

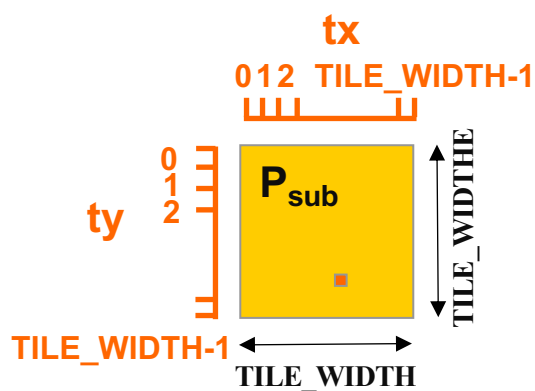
```
// Matrix multiplication on the (CPU) host
void MatMulOnHost(float* M, float* N, float* P, int Width)
{
    for (int i = 0; i < Width; i++)
        for (int j = 0; j < Width; j++) {
            float sum = 0.0;
            for (int k = 0; k < Width; k++) {
                sum += M[i * width + k] * N[k * width + j];
            }
            P[i * Width + j] = sum;
        }
}
```



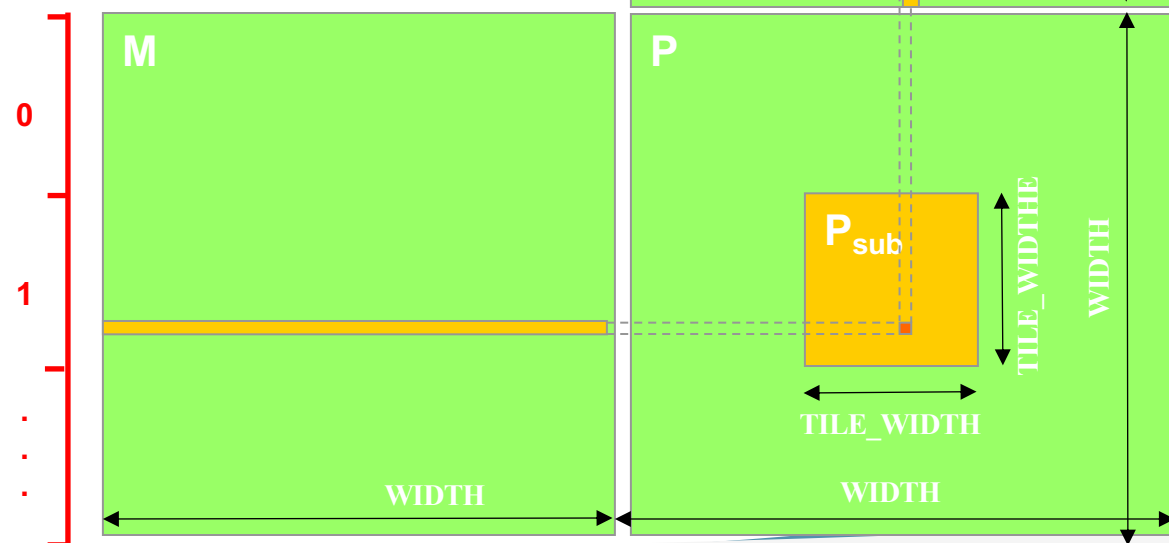


A Tiled Solution

- 将矩阵**P**进行分块，每一块是一个2D的小矩阵，我们称为Tile；
- 将GPU线程组成为2D Block；
- 每一个GPU线程块（Thread Block）计算一个tile。



blockIdx.y





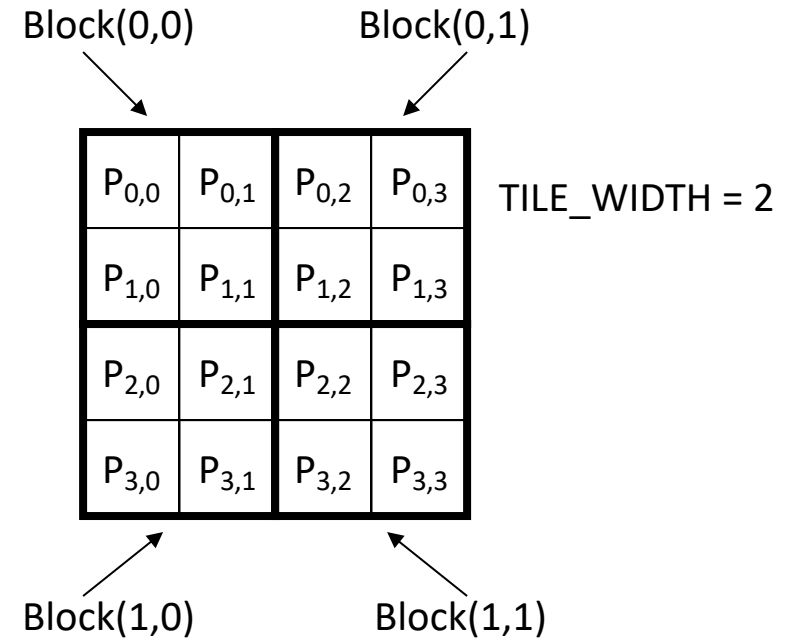
以4x4矩阵为例

WIDTH = 4

TILE_WIDTH = 2

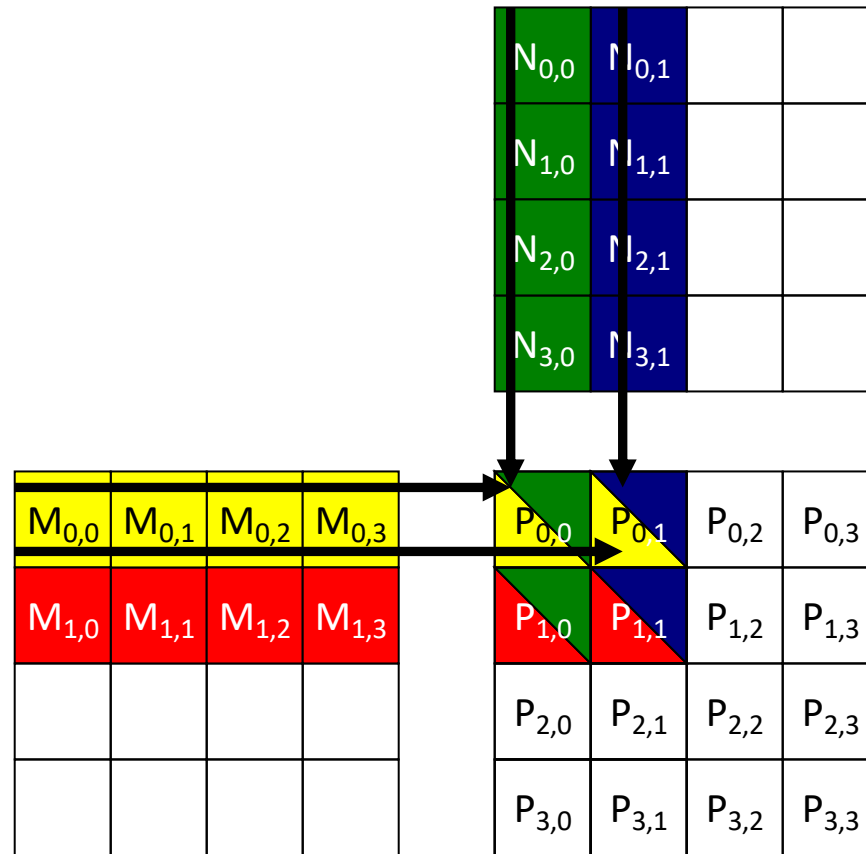
2 x 2 = 4 blocks

Each block has 2 x 2 = 4 threads





以4x4矩阵为例





矩阵乘法Kernel函数

```
__global__ void MatMulKernel(float* Md, float* Nd, float* Pd, int Width)
{
    // Calculate the row index of the P element and M
    int Row = blockIdx.y*blockDim.y + threadIdx.y;
    // Calculate the column index of the P element and N
    int Col = blockIdx.x*blockDim.x + threadIdx.x;

    if ( (Row < Width) && (Col < Width) ) {
        float Pvalue = 0.0;
        // each thread computes one element of the block sub-matrix
        for (int k = 0; k < Width; ++k)
            Pvalue += Md[Row*Width+k] * Nd[k*Width+Col];
        Pd[Row*Width+Col] = Pvalue;
    }
}
```



Kernel函数计时

```
#include <sys/time.h>

double cpuTime() {
    struct timeval tp;
    gettimeofday(&tp, NULL);
    return ( (double)tp.tv_sec + (double)tp.tv_usec * 1.e-6 );
}

double t_start = cpuTime();
/* call your CUDA kernel */
your_kernel<<<grid, block>>>(...);
/* block the host until the GPU completed the kernel function */
cudaDeviceSynchronize();
double kernel_time = cpuTime() - t_start;

printf("The kernel elapsed %f seconds.\n", kernel_time);
```

线程数目的计算

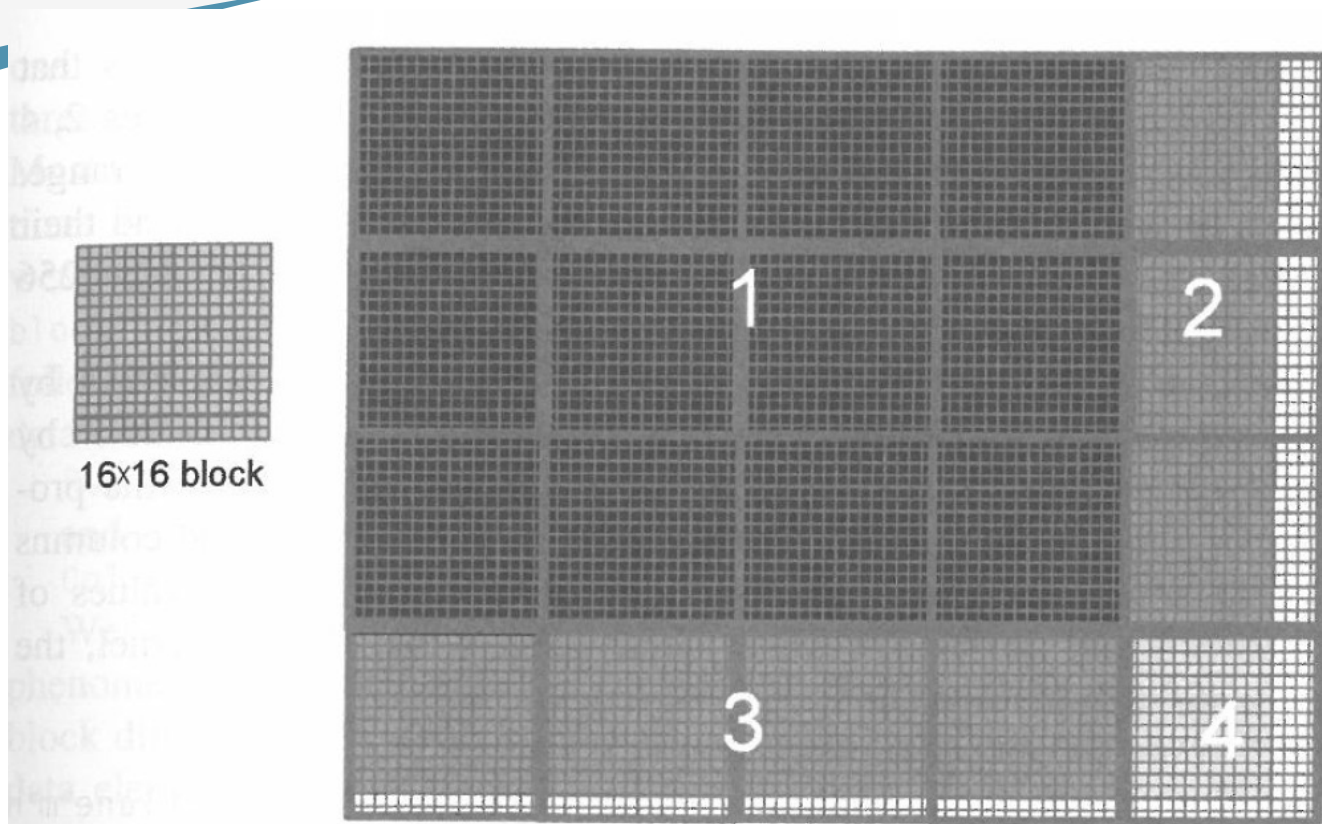


FIGURE 4.5

Covering a 76×62 picture with 16×16 blocks.

图片大小:
 76×62

线程块大小:
 16×16

共需要 5×4 个线程块!

最终生成 80×64 个线程, 其中一些会被浪费。



CPU VS GPU

➤ 测试机器配置

- CPU: Intel Core i7-3770 @3.9GHz
- GPU: Nvidia GT640 with 384 cores @900MHz

Matrix Size	N = 512	N = 1024	N = 1536	N = 2048
GPU time	18.5ms	141ms	470.7ms	1112ms*
CPU time	181.6ms	6817ms	23863ms	67797ms
GPU Flops	14.5G	15.2G	15.4G	15.4G
CPU Flops	1.48G	0.31G	0.30G	0.25G
Speedup	9.8	48.3	50.7	61

*Remark: The running time on Nvidia GTX780 for N = 2048 is 112.7ms !



思考：我们发挥了GPU的所有本领了？

- GTX780拥有2304个ALU小核，那么：
 - 假设它们能够同时进行单精度浮点数计算，理论单精度计算峰值为 $\sim 4\text{T Flops}$
 - 我们代码的性能： $2 \times 2048^3 / 112.7\text{ms} = 152.4\text{GFlops}$
 - 也就是说，才发挥了GPU约3.8%的计算性能
- 瓶颈在哪里？
 - 考虑到GT640的显存带宽是28.5GB/s, GTX780的显存带宽是288GB/s
 - 显存访问是整个程序性能的瓶颈所在！



目录 | CONTENT

- 1 GPU架构与编程模型
- 2 CUDA编程
- 3 GPU访存优化
- 4 GPU控制流和指令优化



CPU与GPU内存拷贝

- `cudaMemcpy()`
 - ❑ 只能用于device <-> host之间的传输，不能用于device <-> device
- CPU与GPU之间的内存拷贝需要经过PCIe连接
 - ❑ PCIe x16 3.0传输带宽为16GB/s
 - ❑ 可通过CUDA SDK中bandwidthTest工具测试PCIe传输带宽和延时
- 在执行kernel之前和之后通常需要进行内存拷贝
 - ❑ 拷贝为线性执行时，有等待时间，性能低下



time →

```
[CUDA Bandwidth Test] - Starting...
Running on...

Device 0: Tesla V100-PCIe-32GB
Quick Mode

Host to Device Bandwidth, 1 Device(s)
PINNED Memory Transfers
Transfer Size (Bytes)      Bandwidth(GB/s)
32000000                  12.3

Device to Host Bandwidth, 1 Device(s)
PINNED Memory Transfers
Transfer Size (Bytes)      Bandwidth(GB/s)
32000000                  13.2

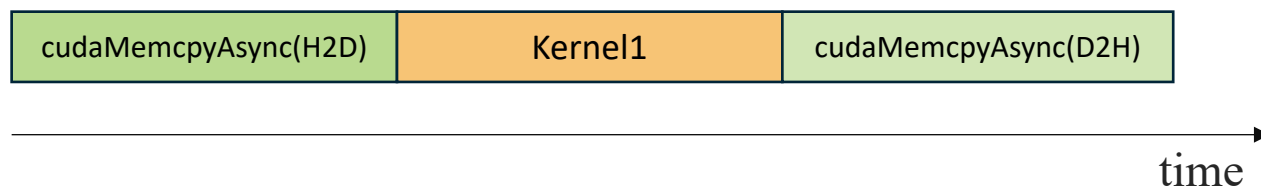
Device to Device Bandwidth, 1 Device(s)
PINNED Memory Transfers
Transfer Size (Bytes)      Bandwidth(GB/s)
32000000                  729.3

Result = PASS
```

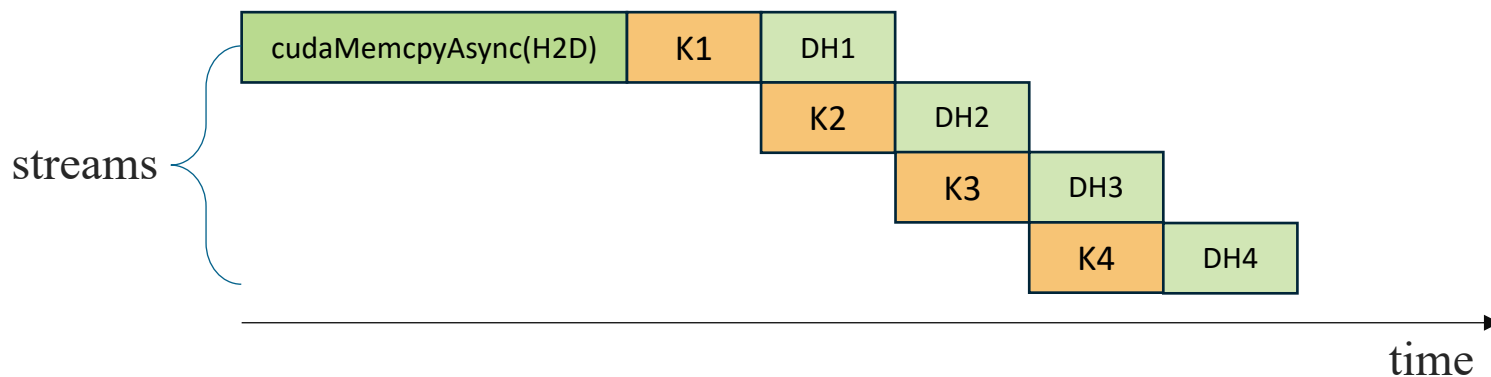


使用Streams并发拷贝

➤ 线性执行



➤ 并发执行：kernel执行与D2H拷贝重叠





使用Streams并发要求

- CUDA操作必须在不同的、非0的streams中
 - ❑ 0号stream即默认的CUDA stream
- cudaMemcpyAsync必须与 “pinned” memory一起用
 - ❑ 即Page-locked memory
 - ❑ 使用cudaMallocHost() 或 cudaHostAlloc()申请的CPU内存
- 有足够的硬件资源
 - ❑ cudaMemcpyAsyncs在不同的方向 (PCIe资源)
 - ❑ 设备资源如共享内存、寄存器等



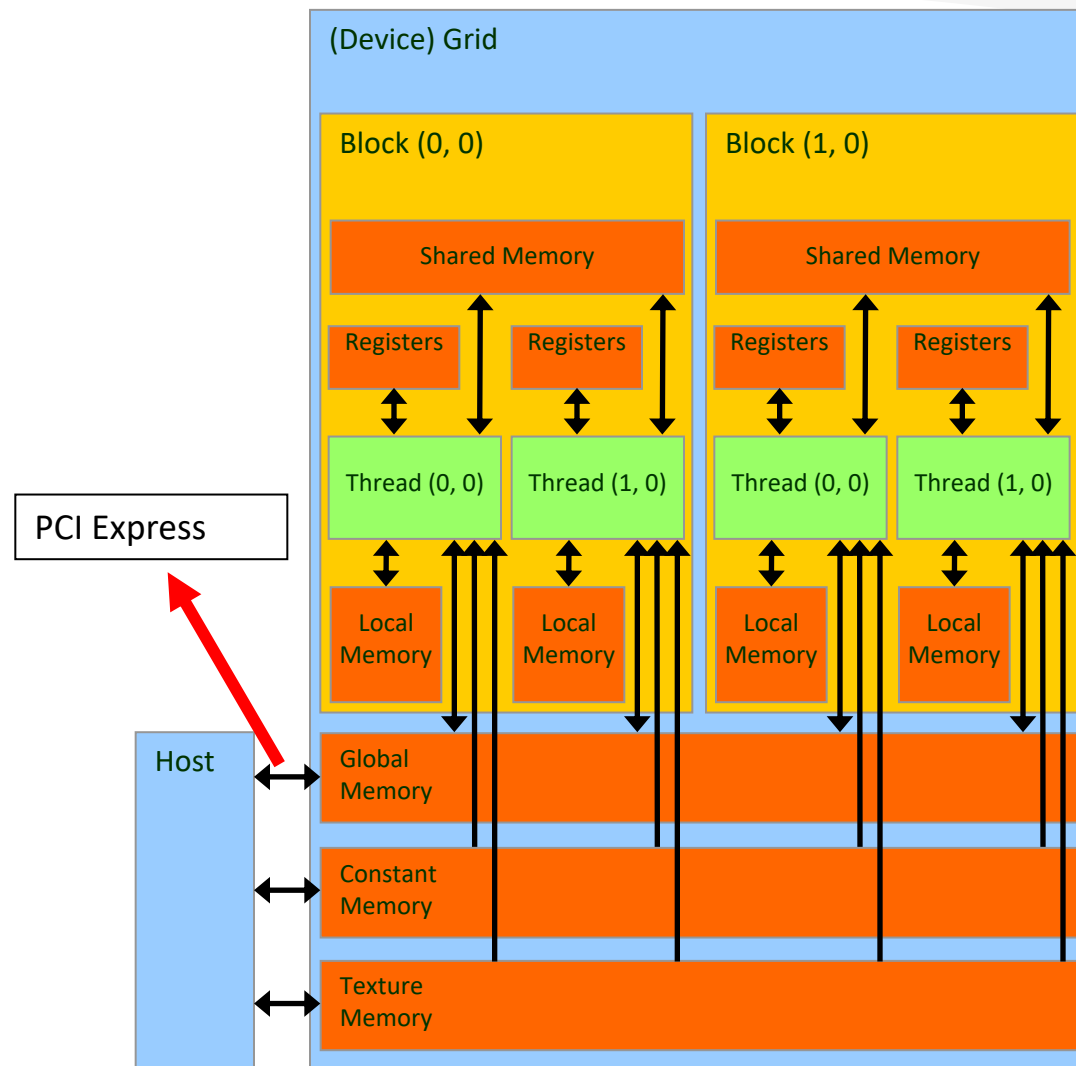
使用Streams并发举例

```
cudaStream_t stream1, stream2, stream3, stream4;  
cudaStreamCreate(&stream1);  
...  
cudaMalloc(&dev1, size);  
cudaMallocHost(&host1, size);  
...  
cudaMemcpyAsync(dev1, host1, size, H2D, stream1);  
kernel2<<<grid, block, 0, stream2>>>(..., dev2, ...);  
kernel3<<<grid, block, 0, stream3>>>(..., dev3, ...);  
cudaMemcpyAsync(host4, dev4, size, D2H, stream4);  
Some_CPU_Methods();
```



GPU存储抽象

- Registers : 线程独立
- Local memory : 线程独立
- Shared memory : Block内所有线程共享
- Global memory : 所有线程共享
- Constant memory : 所有线程共享
- Texture memory : 所有线程共享
- Host memory : 所有线程共享





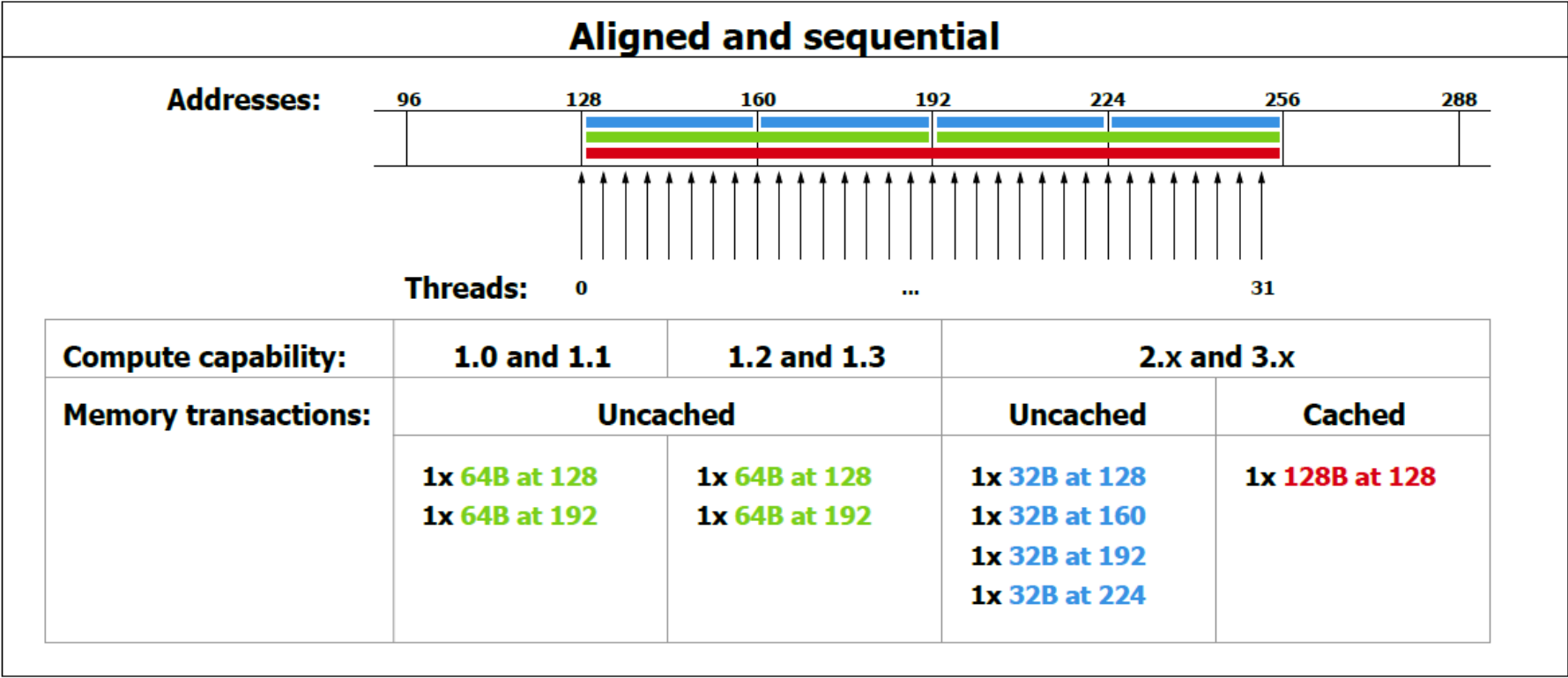
全局内存访问优化

- 内存读写操作的发射是以warp（32线程）为单位
 - 即1条指令，32个数据访问
 - 如何满足在1条指令得到32个数据？
- 对齐访问（Coalescing）
 - 当内存访问满足一定条件时，Global memory的读或写可以一个Transaction完成
 - 一次访问将128字节（计算能力2.x+）放入L1 Cache line中
 - 一个warp的32线程内存访问请求Cache line
 - 一次访问将32字节（计算能力3.x+和5.x+）放入L2 Cache line中



对齐访问举例（1）

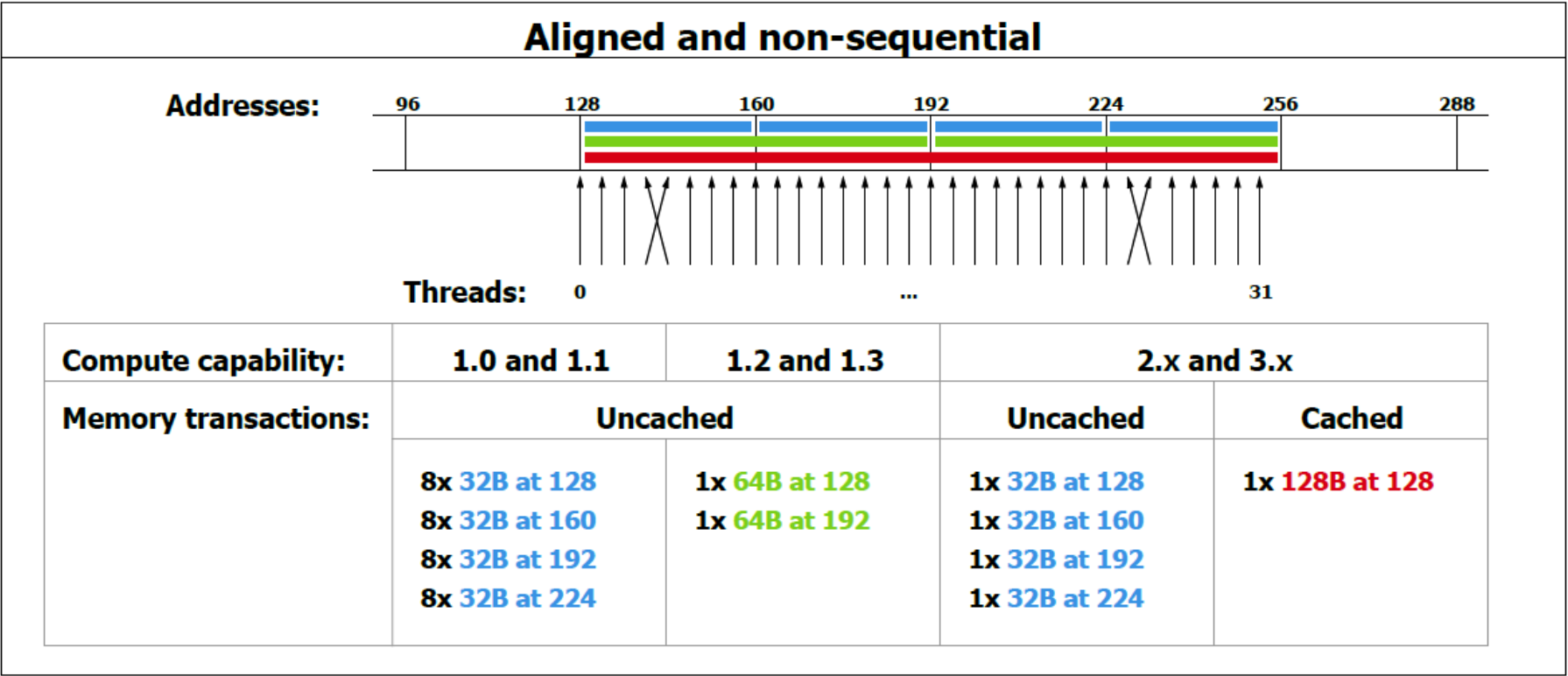
➤ 对齐且顺序访问：每个线程读取4字节（一个warp为32线程）





对齐访问举例（2）

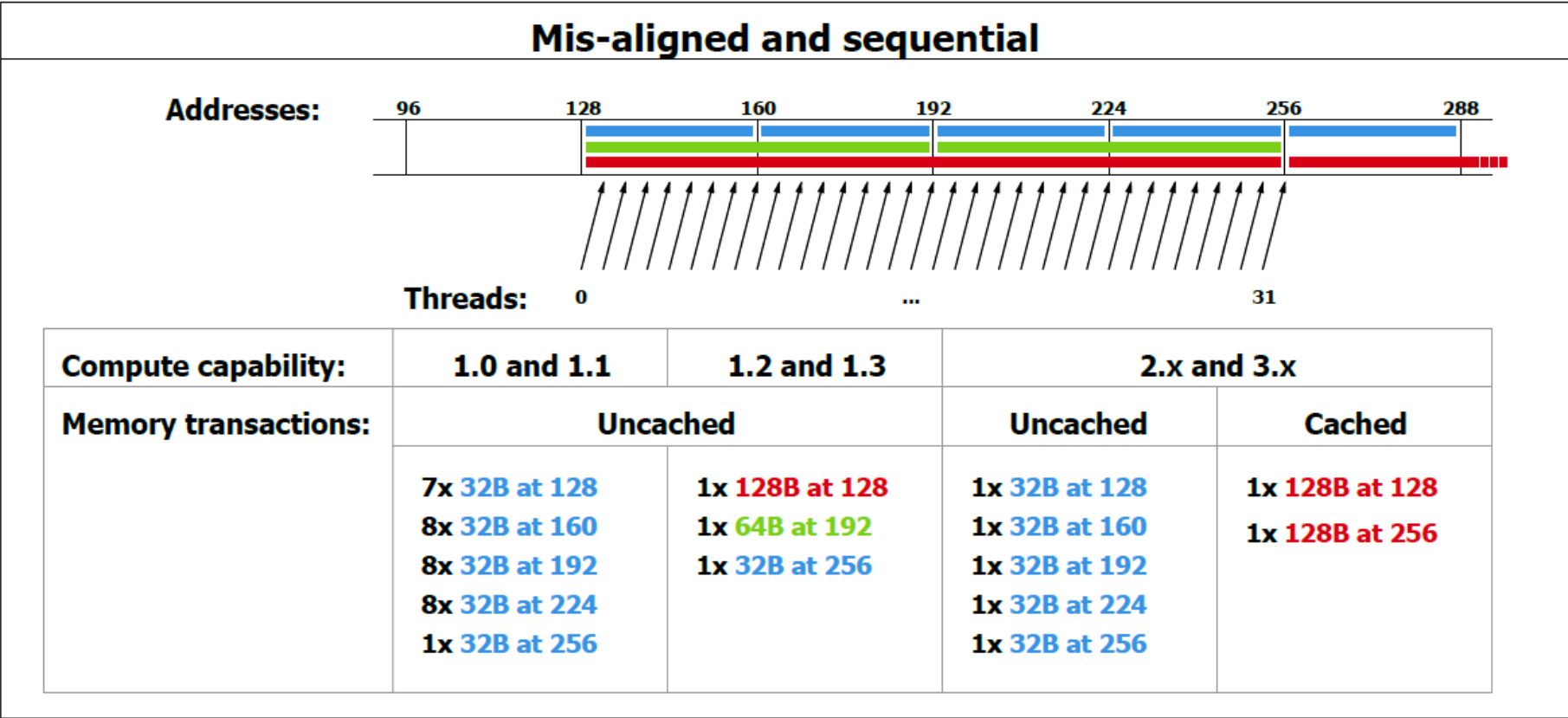
➤ 对齐但非顺序访问：每个线程读取4字节（一个warp为32线程）





对齐访问举例（3）

➤ 非对齐且非顺序访问：每个线程读取4字节（一个warp为32线程）





共享内存访问优化

- GPU上共享内存 (Shared memory)
 - 物理上与L1 Cache相同
 - 属于片上存储器
 - 访问速度接近寄存器，远快于Global memory
- 用处
 - 可用于Block内线程的通信
 - 可做为Global memory的高速缓存并可编程控制
- 组织结构
 - 划分为等大小的内存模块，称为Bank
 - 相邻的4字节或8字节的字可能位于不同的Bank (取决不同计算能力)

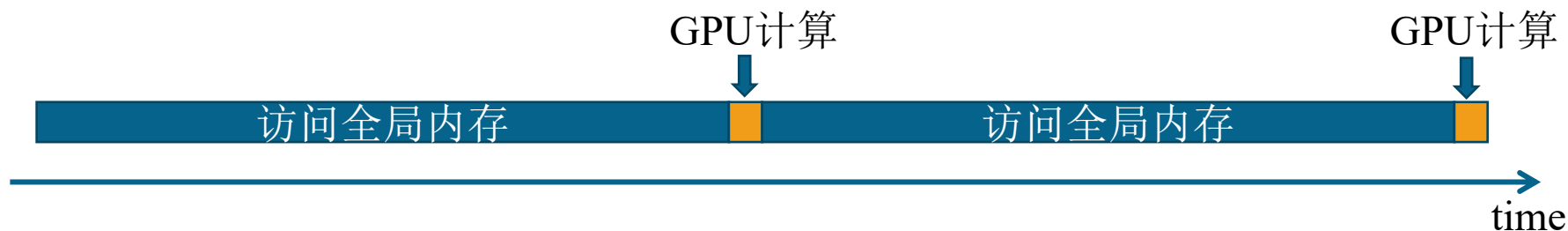


分析矩阵乘法的瓶颈

```
/* 内核函数的关键部分 */  
for (int k = 0; k < Width; ++k)  
    Pvalue += Md[Row*Width+k] * Nd[k*Width+Col];
```



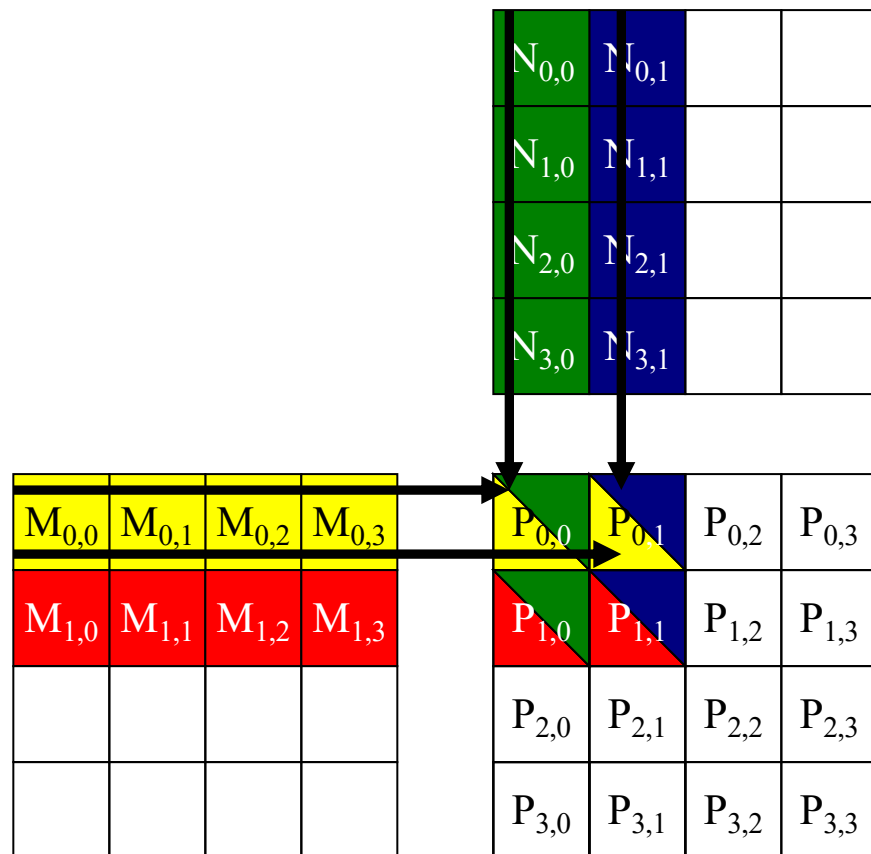
- 前文内核代码的 CGMA 值为 1
- 每次计算都需要从全局内存访问一次数据
- 因此，性能受到内存带宽的限制
 - 大多数时候，GPU 都无事可做！





全局内存 VS 共享缓存

- 假设线程块大小为 $n \times n$ 。矩阵大小为 $N \times N$
- 之前的方案中每个线程块需进行 $2n^2N$ 次全局内存访问
- 利用共享内存，在计算时单个线程块时可只装载矩阵 M 的 n 行数据，矩阵 N 的 n 列数据，只需访问 $2nN$ 次全局内存
- 可将全局内存访问量减少 n 倍！





共享缓存的挑战

- 共享内存的大小有限
- 同时将矩阵M的n行和矩阵N中的n列加载到共享内存是不现实
- 解决方法：
 - 将n行和n列划分为多个tile，这样可按tile将矩阵M和矩阵N加载到共享内存中
 - 计算共享内存中两个tiles
 - 将新的矩阵M和矩阵N的tiles加载到共享内存，然后重复计算直到完成



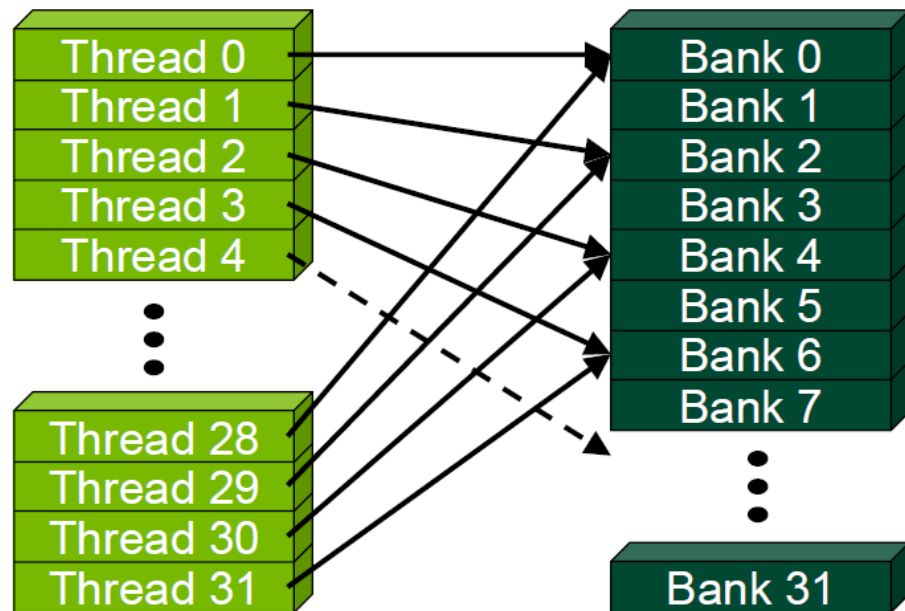
共享内存Bank

- Compute Capability 1.x
 - ❑ 16 banks, 4-byte wide
- Compute Capability 2.x
 - ❑ 32 banks, 4-byte wide
- Compute capability 3.x
 - ❑ 32-bit mode: 32 banks, 4-byte wide
 - ❑ 64-bit mode: 32 banks, 8-byte wide
- Compute capability 5.x
 - ❑ 32 banks, 4-byte wide
- Compute capability 7.x
 - ❑ 32 banks, 4-byte wide

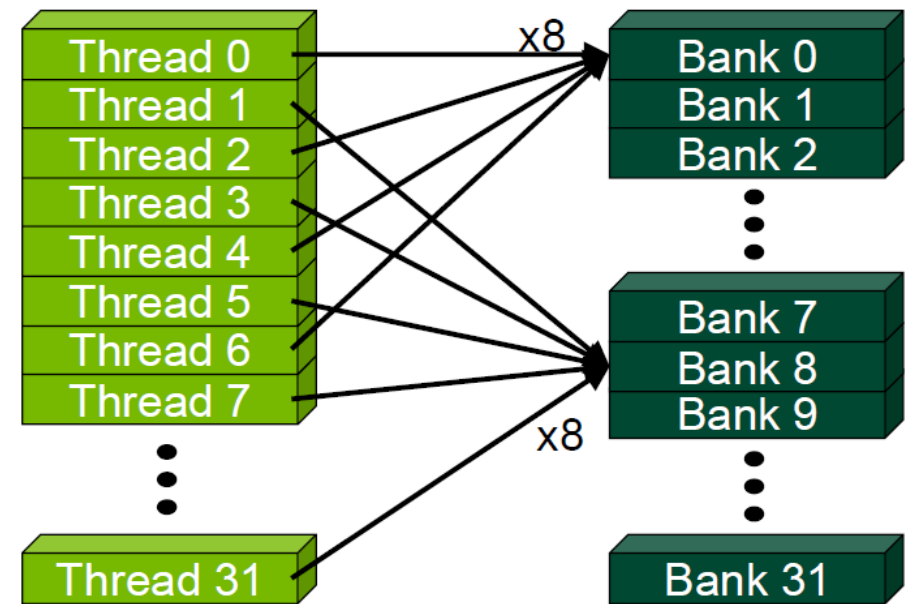


共享内存Bank冲突

- 共享内存访问是以warp为单位（即32线程）发射请求的
- Bank冲突：一个warp中的32个线程中，不同线程（假设为 n ）访问了同一个Bank的共享内存，则需要 n 次共享内存访问必须线性执行



2路Bank冲突

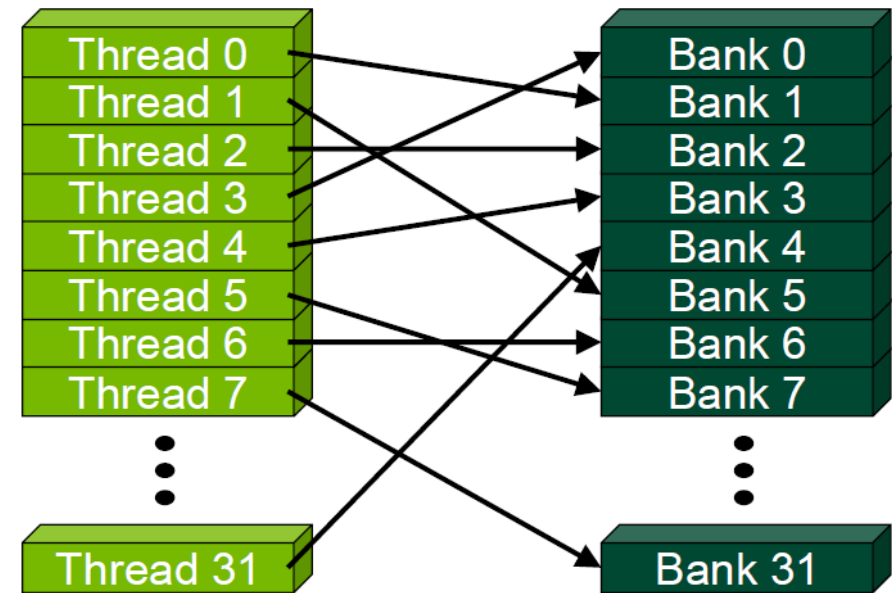
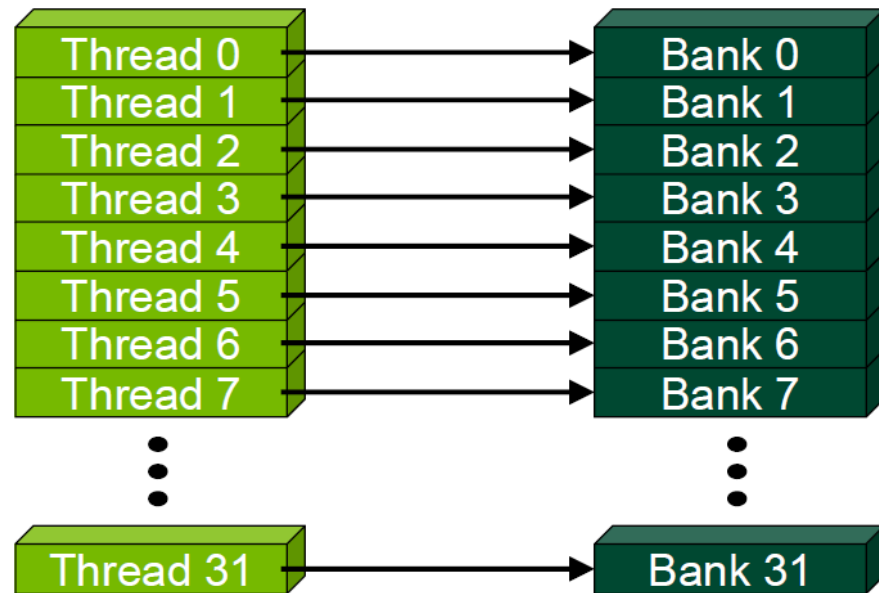


8路Bank冲突



共享内存Bank冲突

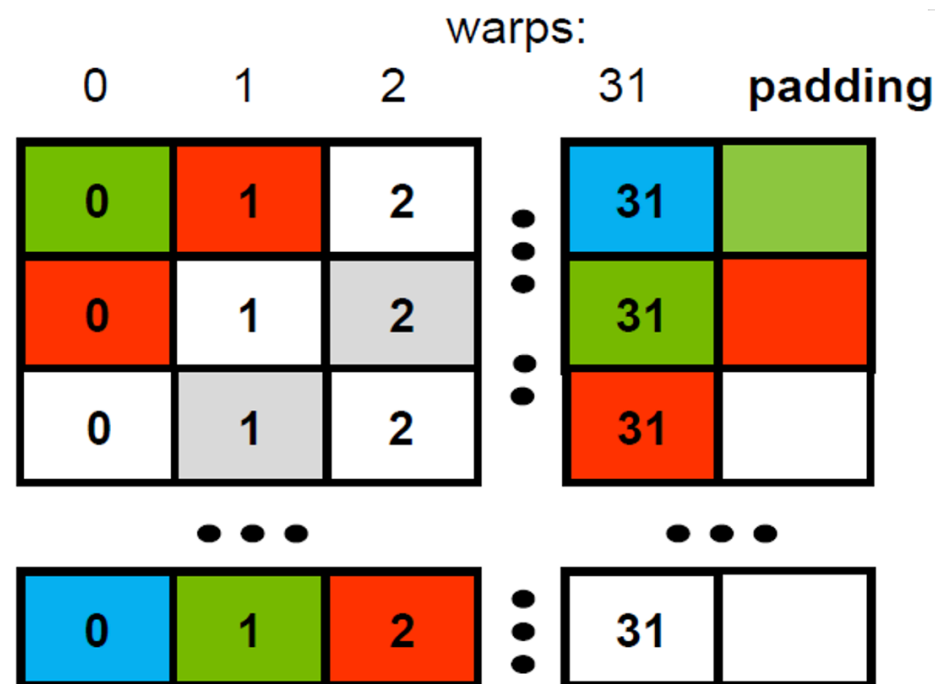
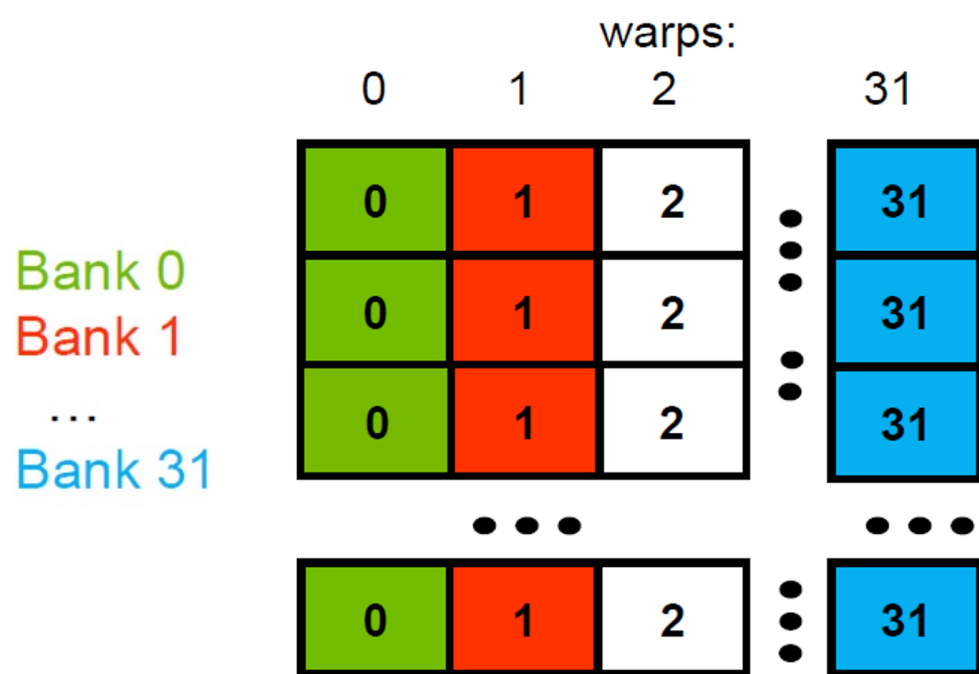
➤ 不存在Bank冲突





避免共享内存Bank冲突

- 32x32的共享内存数组，每个warp访问一列
 - ❑ 32路Bank冲突
- 加一列做为Padding来避免冲突





目录 | CONTENT

- 1 GPU架构与编程模型
- 2 CUDA编程
- 3 GPU访存优化
- 4 GPU控制流和指令优化



GPU条件控制流

➤ 分支处理：性能很大差别

- ❑ 在一个warp里的所有线程（32）执行不同分支
- ❑ 不同分支为线性执行

➤ 常见的情况

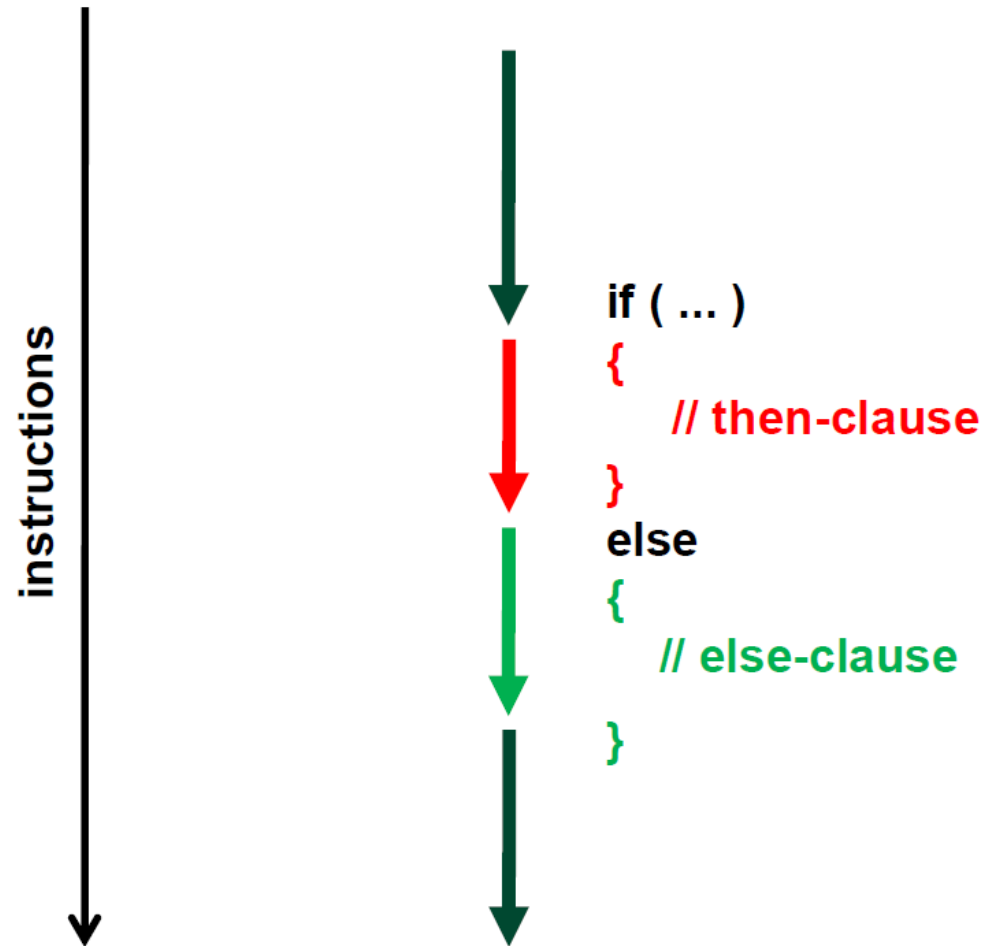
❑ 分支发散例子

- `if (threadIdx.x > 2) { path1; } else { path2; }`
- 0和1线程执行path1时，2-31线程等待；2-31线程执行path2时，0和1线程等待

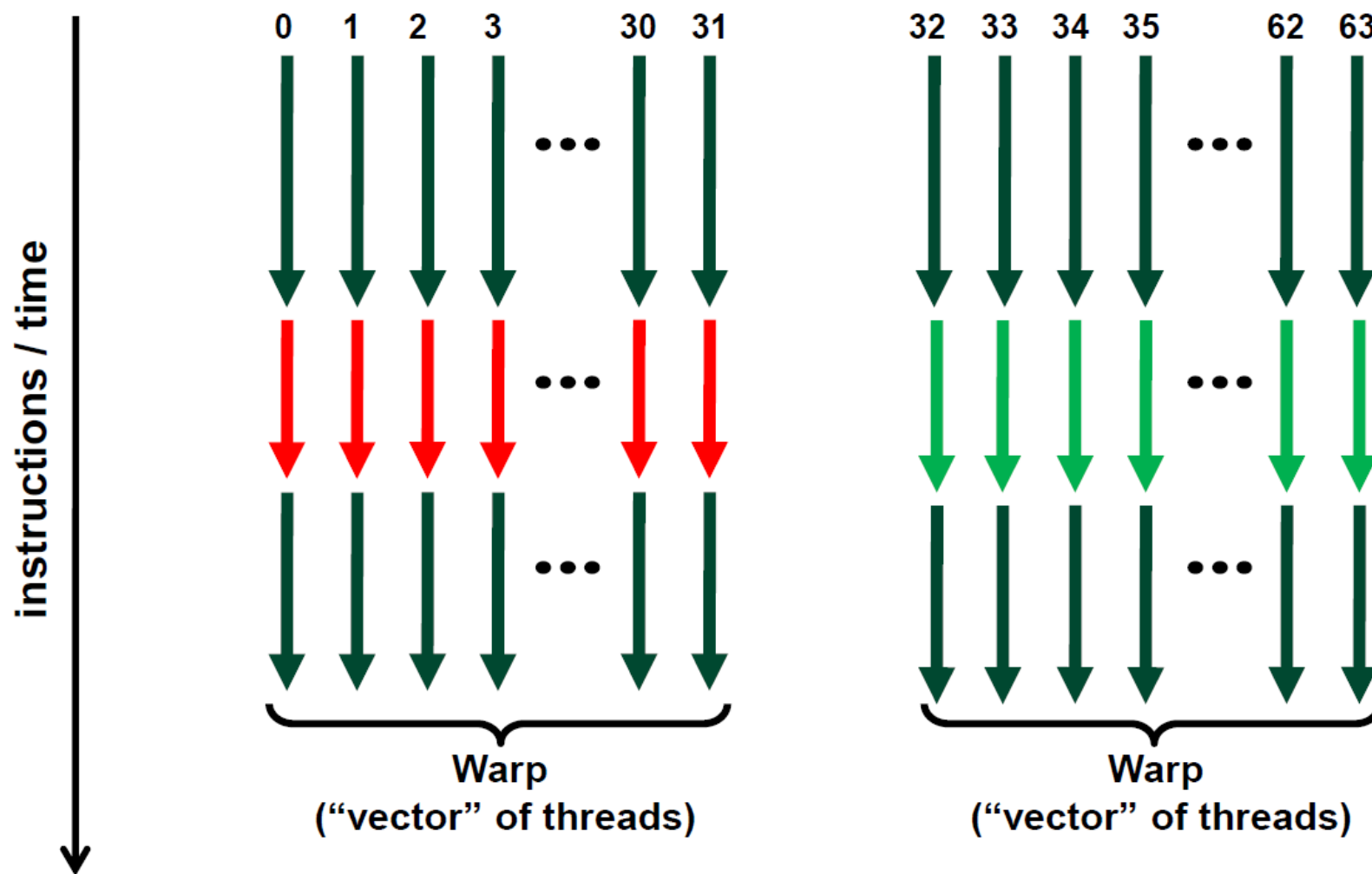
❑ 分支不发散例子

- `if (threadIdx.x / WARP_SIZE > 2) { path1; } else { path2; }`
- 每个warp的所有线程都只执行一个分支

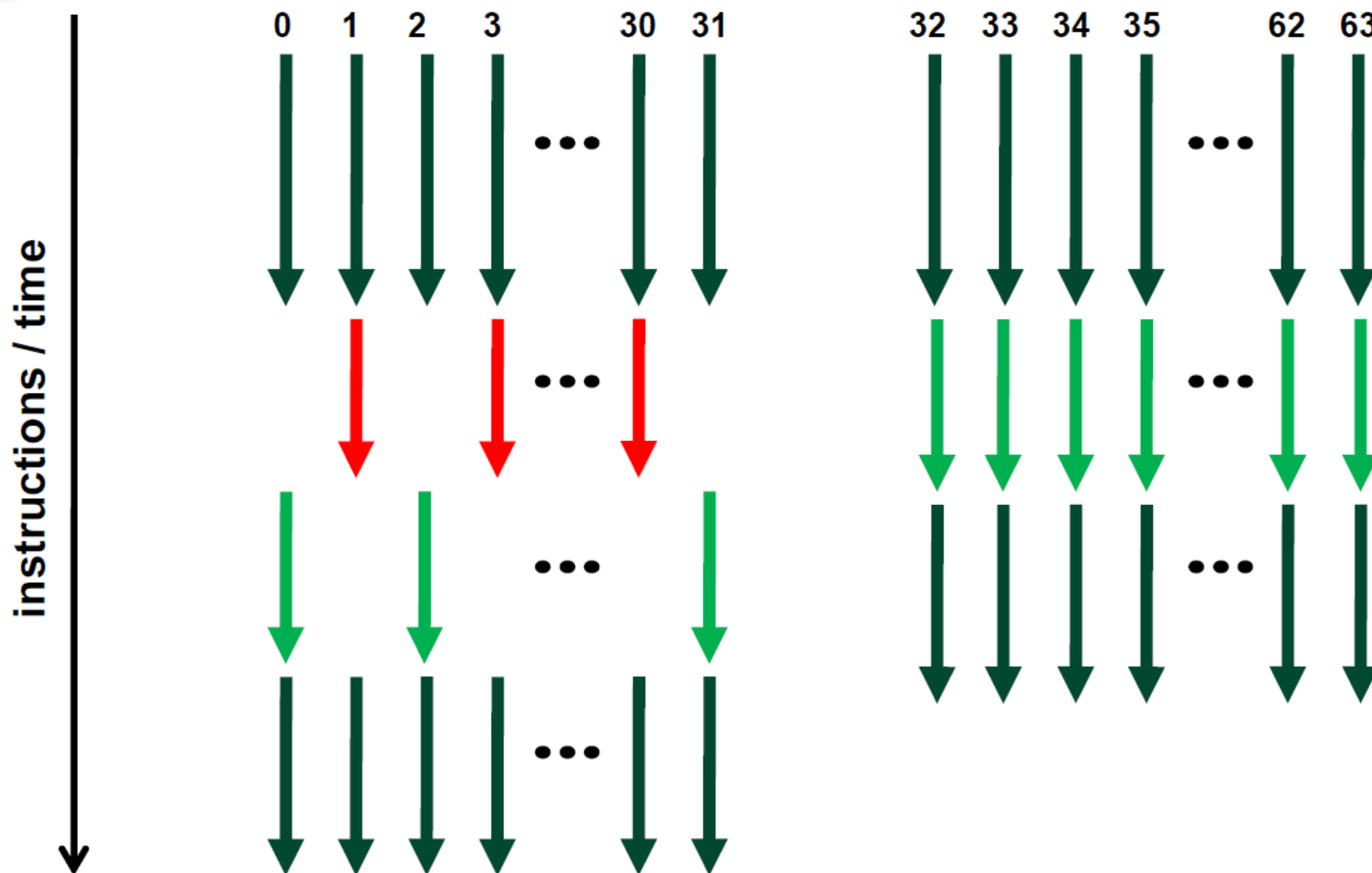
GPU条件控制流



GPU条件控制流



GPU条件控制流





授课教师：王强

邮箱：qiang.wang@hit.edu.cn

谢谢

Thank You